Introduction to Statistical Learning with Applications

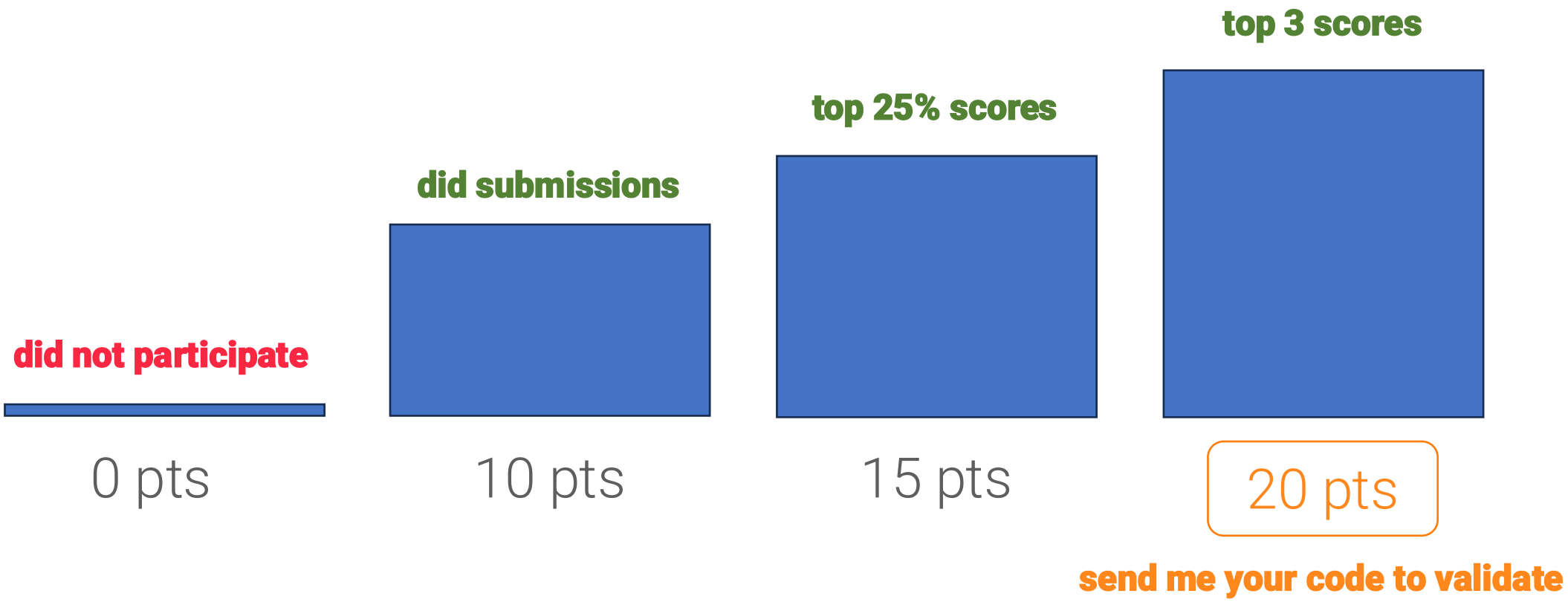
CM7: Decision trees

Pedro L. C. Rodrigues

About challenge #1

⚠️ There was a problem with the challenge : **it was possible to find y_test online** 🤡

Our solution ensure grading as fair as possible (in less than ideal circumstances)



Introduction to Statistical Learning with Applications

CM7: Decision trees

Pedro L. C. Rodrigues

Discriminative approaches

There are many strategies for approximating the Bayes classifier.

$$f(\mathbf{x}_0) = \underset{k}{\operatorname{argmax}} \operatorname{Prob}\Big(Y = k \mid \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big)$$

We have seen so far:

- K-nearest neighbours (kNN)
- Logistic regression

Discriminative approaches

- Linear and quadratic discriminant analysis
- Naïve Bayes classifier

Generative approaches

In the following lectures, we will see

- Random forests
- Gradient boosted trees

Ensembling approaches

Decision trees

There are many strategies for approximating the Bayes classifier.

- Random forests
- Gradient boosted trees

Ensembling approaches

Both rely on **decision trees** and can be used for regression and classification

Example: The Titanic survival dataset

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
5	6	0	3	Moran, Mr. James	male	NaN	0	0	330877	8.4583	NaN	Q
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S

Decision trees

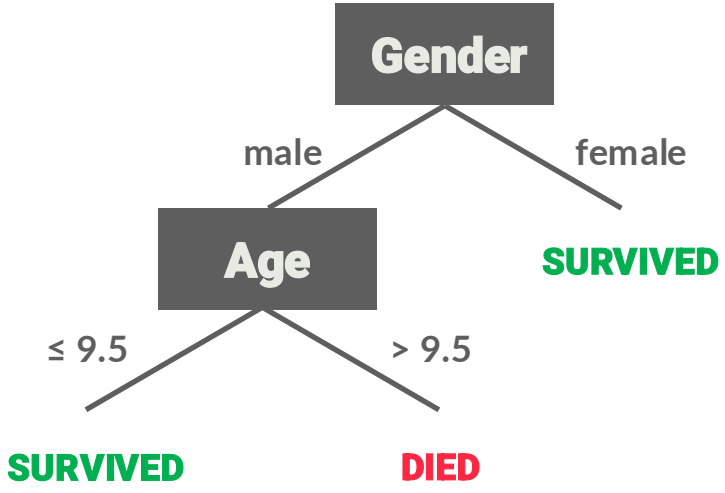
There are many strategies for approximating the Bayes classifier.

- Random forests
- Gradient boosted trees

Ensembling approaches

Both rely on **decision trees** and can be used for regression and classification

Example: The Titanic survival dataset

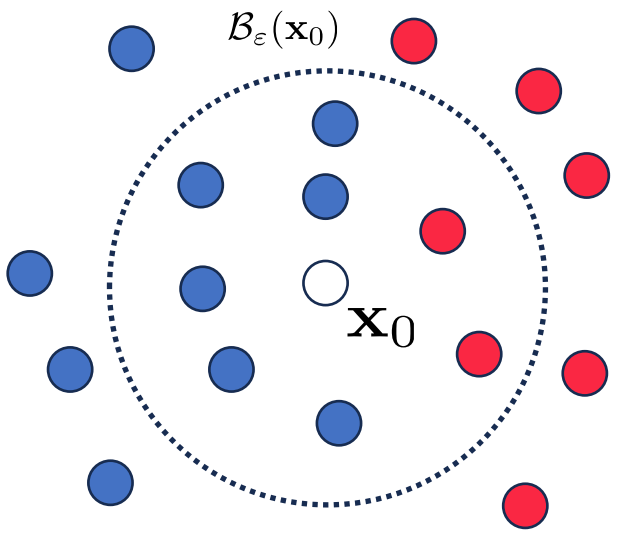


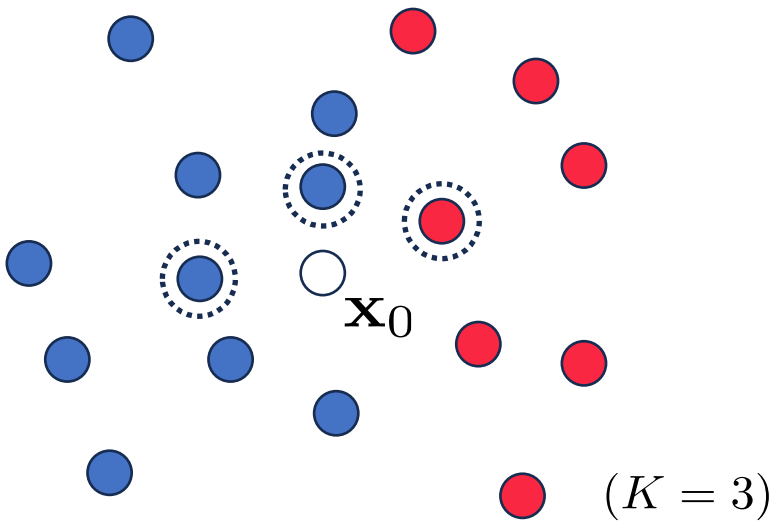
Decision trees

But first, we will talk again about k-nearest neighbours (kNN)

In kNN we approximate $\mathcal{B}_\varepsilon(\mathbf{x}_0) \approx \mathcal{N}_K(\mathbf{x}_0)$ i.e. the set of K nearest neighbours of $\mathbf{x}_0$

$$\mathrm{Prob}\Big(Y = 1 \,\Big|\, \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x}_0)} \mathbf{1}_{y=1}(y_i)$$





Decision trees

But first, we will talk again about k-nearest neighbours (kNN)

In kNN we approximate $\mathcal{B}_\varepsilon(\mathbf{x}_0) \approx \mathcal{N}_K(\mathbf{x}_0)$ i.e. the set of K nearest neighbours of $\mathbf{x}_0$

$$\mathrm{Prob}\Big(Y = 1 \,\Big|\, \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x}_0)} \mathbf{1}_{y=1}(y_i)$$

- kNN is a nonparametric classifier: it makes no assumptions about the data
  - ↳ It can be very flexible, but sub-optimal when we know something about the data
- It works well in low dimensions, but suffers from the **curse of dimensionality**
- The **hyper-parameter** K is usually chosen via cross-validation

Decision trees

But first, we will talk again about k-nearest neighbours (kNN)

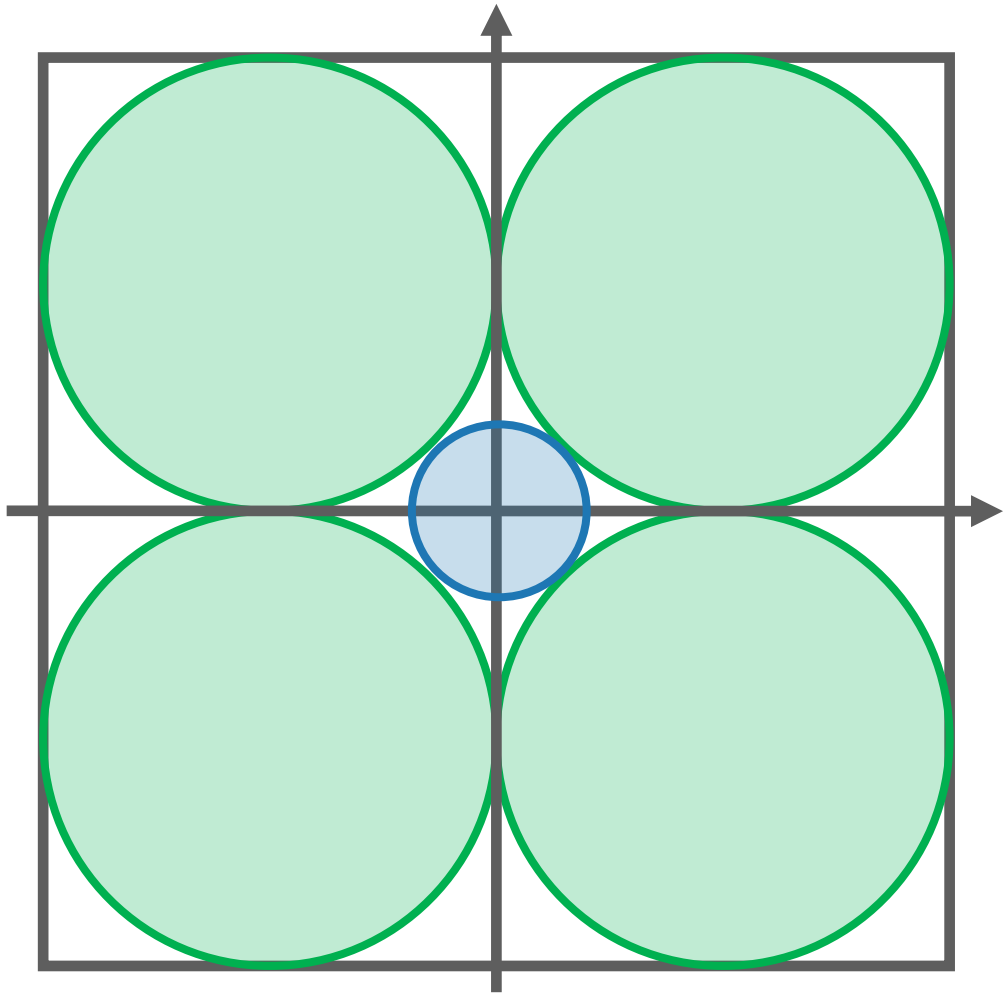
In kNN we approximate $\mathcal{B}_\varepsilon(\mathbf{x}_0) \approx \mathcal{N}_K(\mathbf{x}_0)$ i.e. the set of K nearest neighbours of $\mathbf{x}_0$

$$\mathrm{Prob}\Big(Y = 1 \,\Big|\, \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x}_0)} \mathbf{1}_{y=1}(y_i)$$

- kNN is a nonparametric classifier: it makes no assumptions about the data
  - ↳ It can be very flexible, but sub-optimal when we know something about the data
- It works well in low dimensions, but suffers from the **curse of dimensionality**
- The **hyper-parameter** K is usually chosen via cross-validation

Decision trees

- It works well in low dimensions, but suffers from the curse of dimensionality



Example from JM Steele "The Cauchy-Schwarz master class"

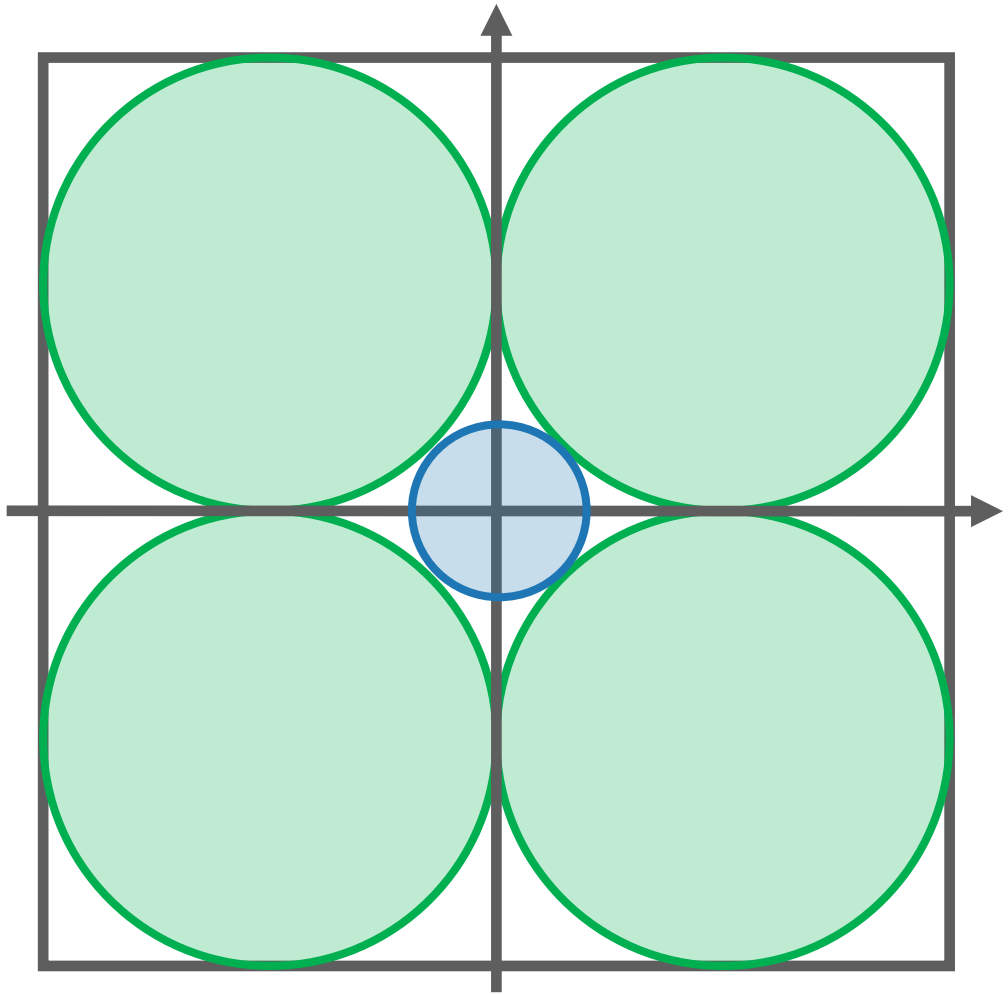
<u>Example of failure of intuition in high-dimensions</u>

In two-dimensional space R^2

- Four unit-radius circles inside $[-2, 2]^2$
- Circle of radius r centred at the origin, confined by the unit-radius circles
- Distance from origin to centers: $\sqrt{2}$
- Radius $r = \sqrt{2} - 1$

Decision trees

- It works well in low dimensions, but suffers from the curse of dimensionality



Example from JM Steele "The Cauchy-Schwarz master class"

<u>Example of failure of intuition in high-dimensions</u>

In d-dimensional space R^d

- 2^d unit-radius circles inside $[-2, 2]^d$
- Sphere of radius r centred at the origin, confined by the unit-radius spheres
- Distance from origins to centers: $\sqrt{d}$
- Radius $r = \sqrt{d} - 1$

But for $d \geq 10$ we have $r > 2$ 🤯

Decision trees

- It works well in low dimensions, but suffers from the curse of dimensionality

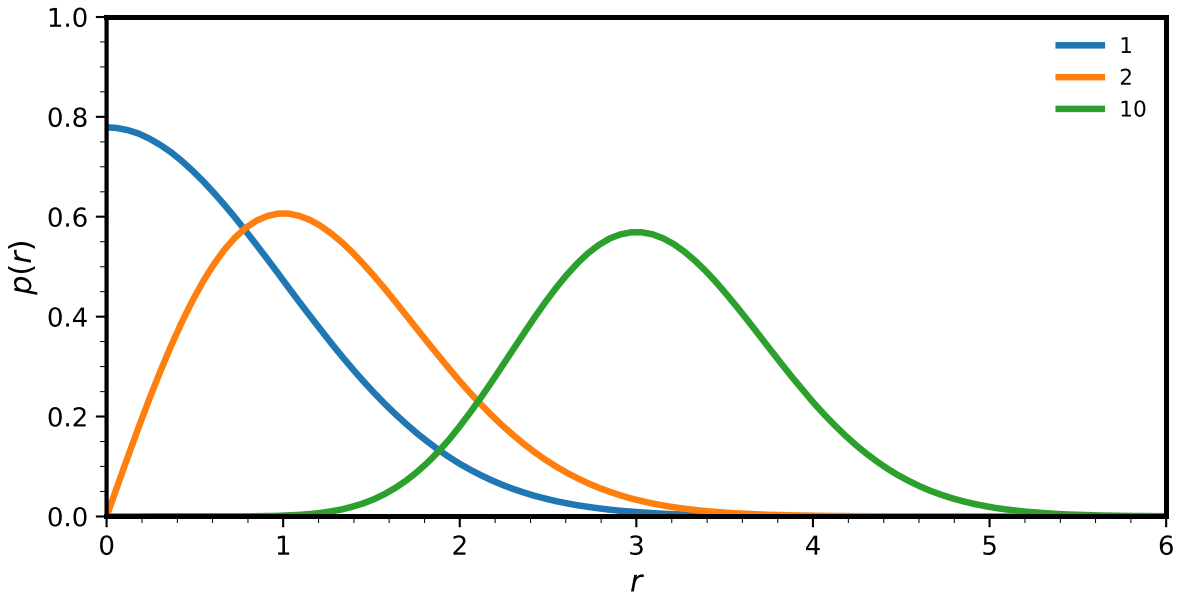
The Normal distribution in high-dimensions

$$p(x_1, \ldots, x_d) \propto \exp\left(-\frac{1}{2}\|\mathbf{x}\|^2\right)$$

Passing to polar coordinates

$$p(r) \propto r^{d-1} \exp\left(-\frac{1}{2}r^2\right)$$

-- This is the marginal that we can get integrating on the angles



Note that the pdf of the radius concentrates quite far from the origin!

Decision trees

But first, we will talk again about k-nearest neighbours (kNN)

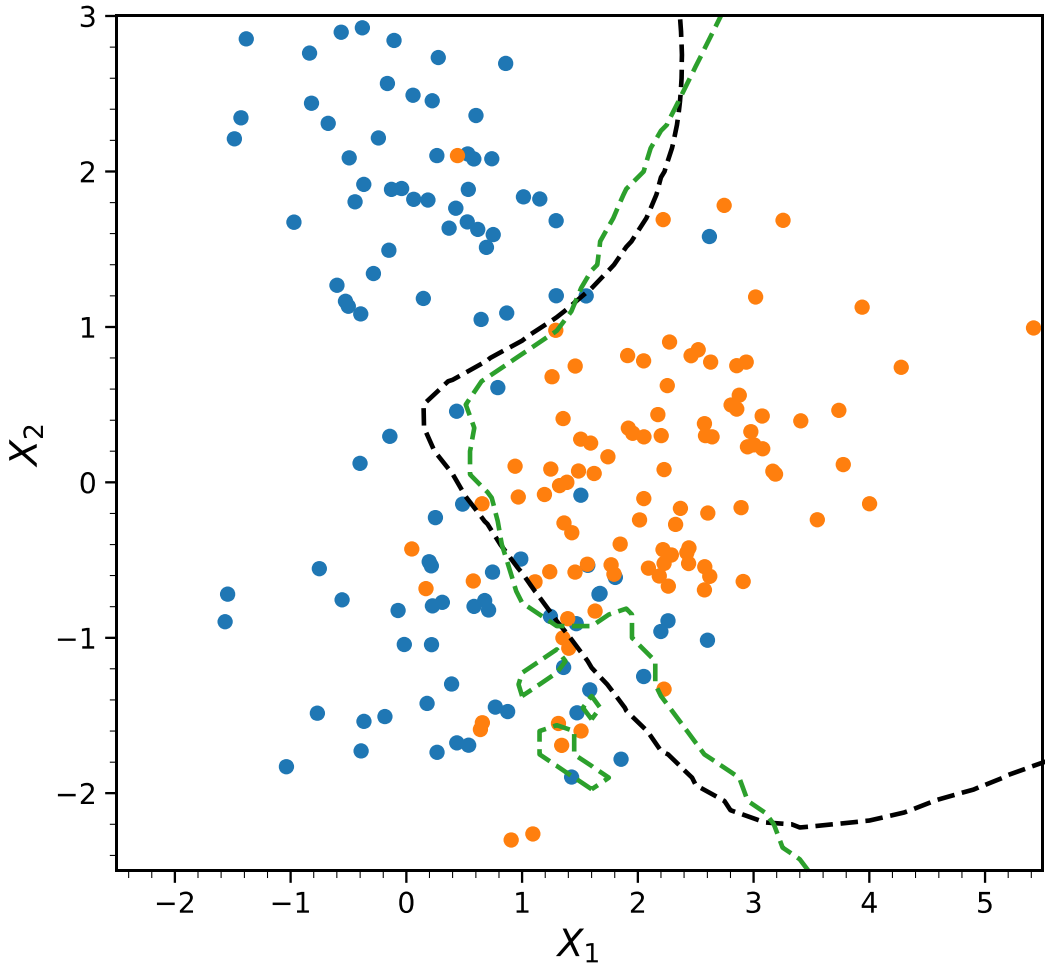
In kNN we approximate $\mathcal{B}_\varepsilon(\mathbf{x}_0) \approx \mathcal{N}_K(\mathbf{x}_0)$ i.e. the set of K nearest neighbours of $\mathbf{x}_0$

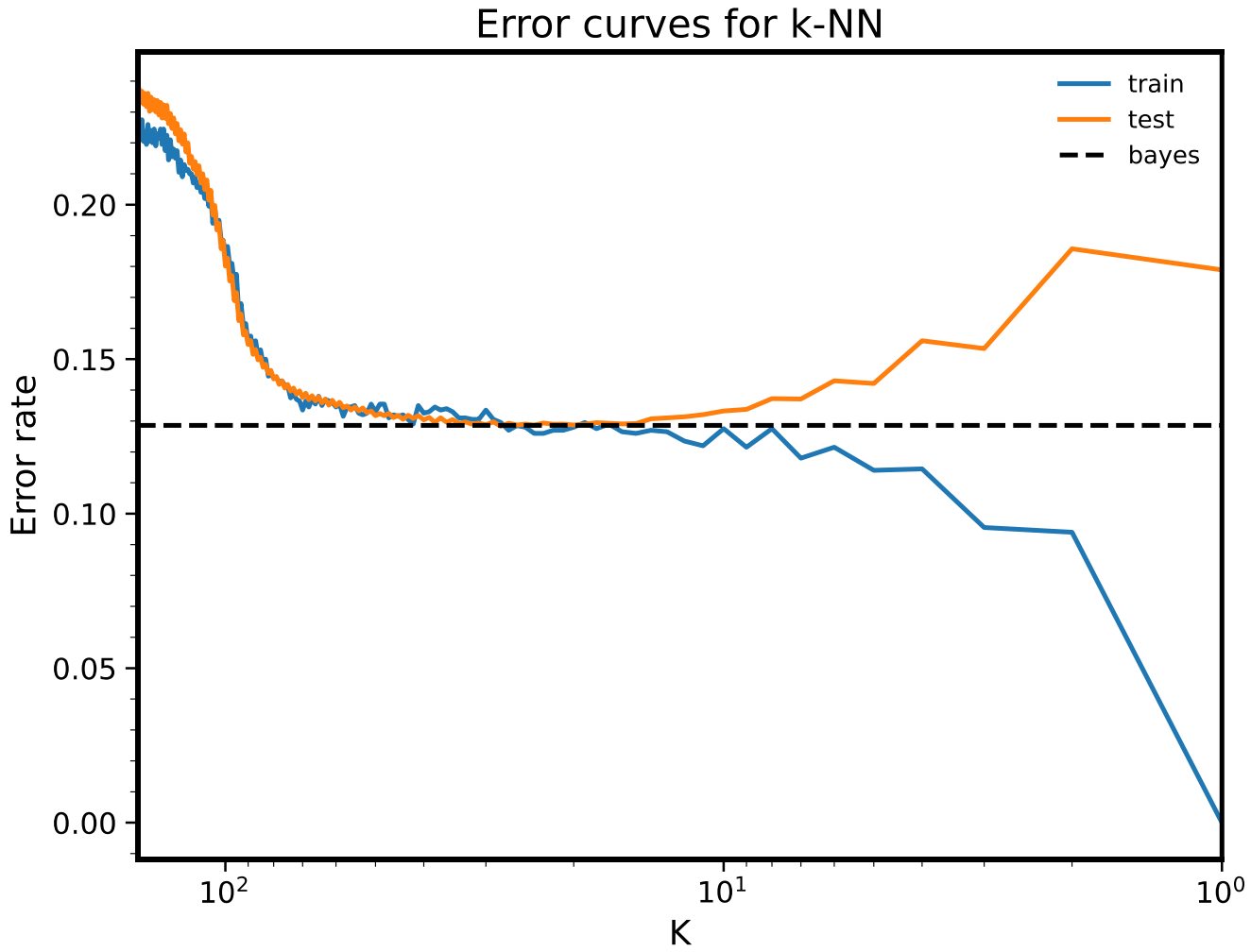
$$\mathrm{Prob}\Big(Y = 1 \,\Big|\, \mathbf{X} \in \mathcal{B}_\varepsilon(\mathbf{x}_0)\Big) \approx \frac{1}{K} \sum_{\mathbf{x}_i \in \mathcal{N}_K(\mathbf{x}_0)} \mathbf{1}_{y=1}(y_i)$$

- kNN is a nonparametric classifier: it makes no assumptions about the data
  - ↳ It can be very flexible, but sub-optimal when we know something about the data
- It works well in low dimensions, but suffers from the **curse of dimensionality**
- The **hyper-parameter** K is usually chosen via cross-validation

Decision trees

- The **hyper-parameter** K is usually chosen via cross-validation

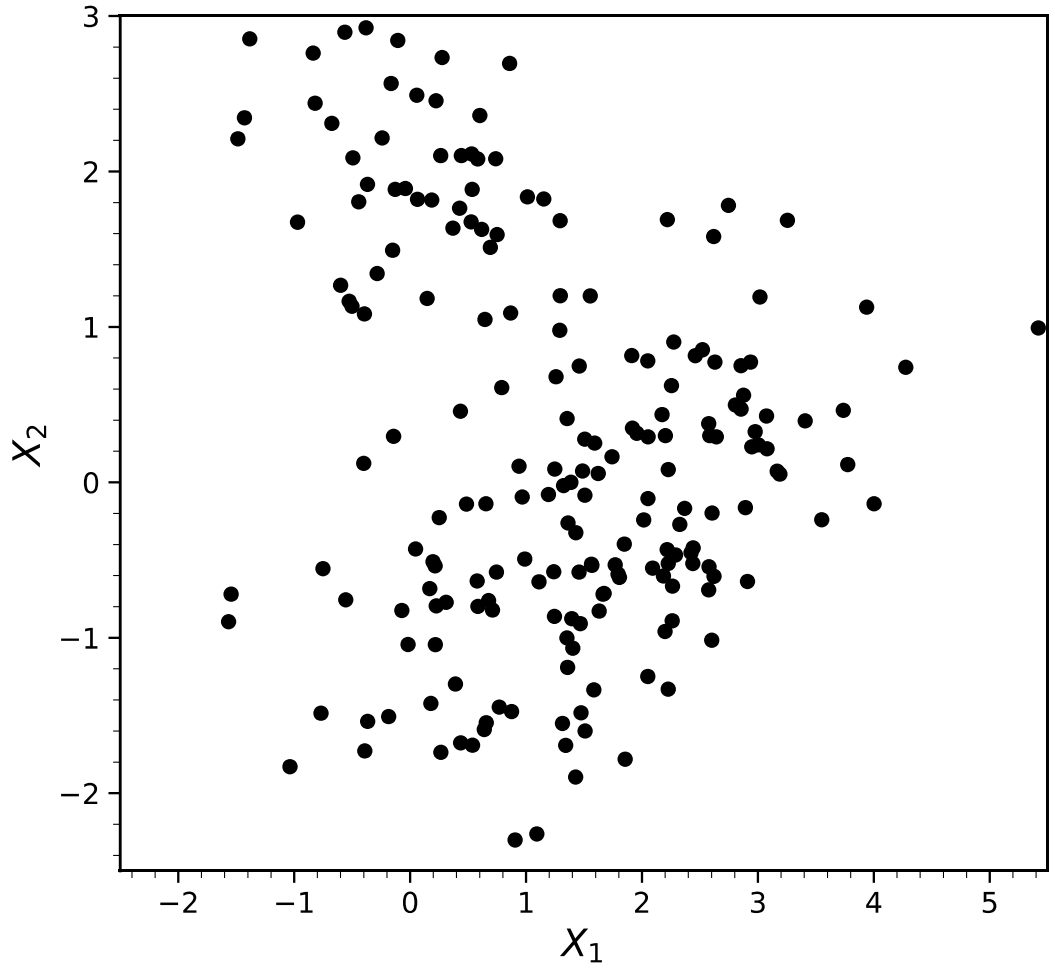




Decision trees

But first, we will talk again about K-nearest neighbours (kNN)

Q: How exactly do we classify a point?



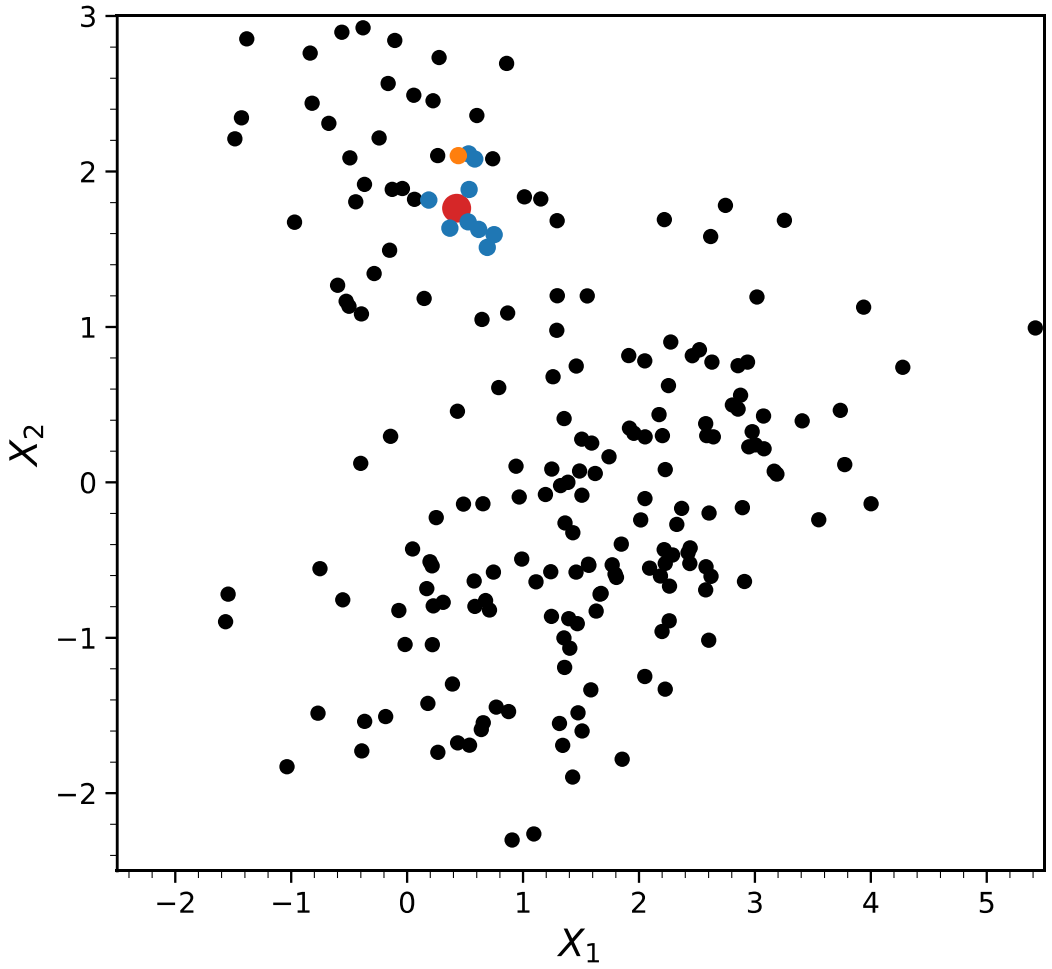
Decision trees

But first, we will talk again about K-nearest neighbours (kNN)

Q: How exactly do we classify a point?

(1) For each point find its K nearest neighbours

(2) Assign the class with most representatives



Decision trees

But first, we will talk again about K-nearest neighbours (kNN)

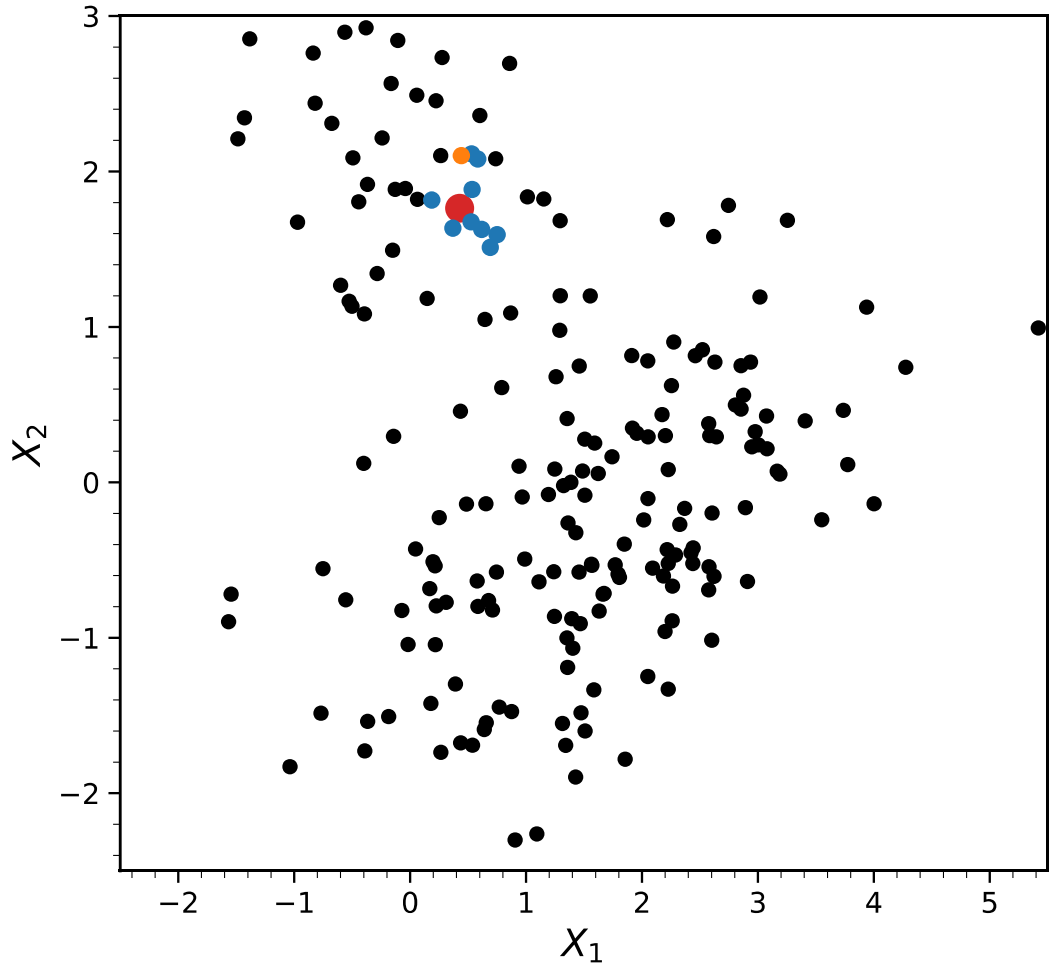
Q: How exactly do we classify a point?

(1) For each point find its K nearest neighbours

(2) Assign the class with most representatives

For N samples, do N(N-1) operations $\mathcal{O}(N^2)$

Can we do better?



Decision trees

But first, we will talk again about K-nearest neighbours (kNN)

Q: How exactly do we classify a point?

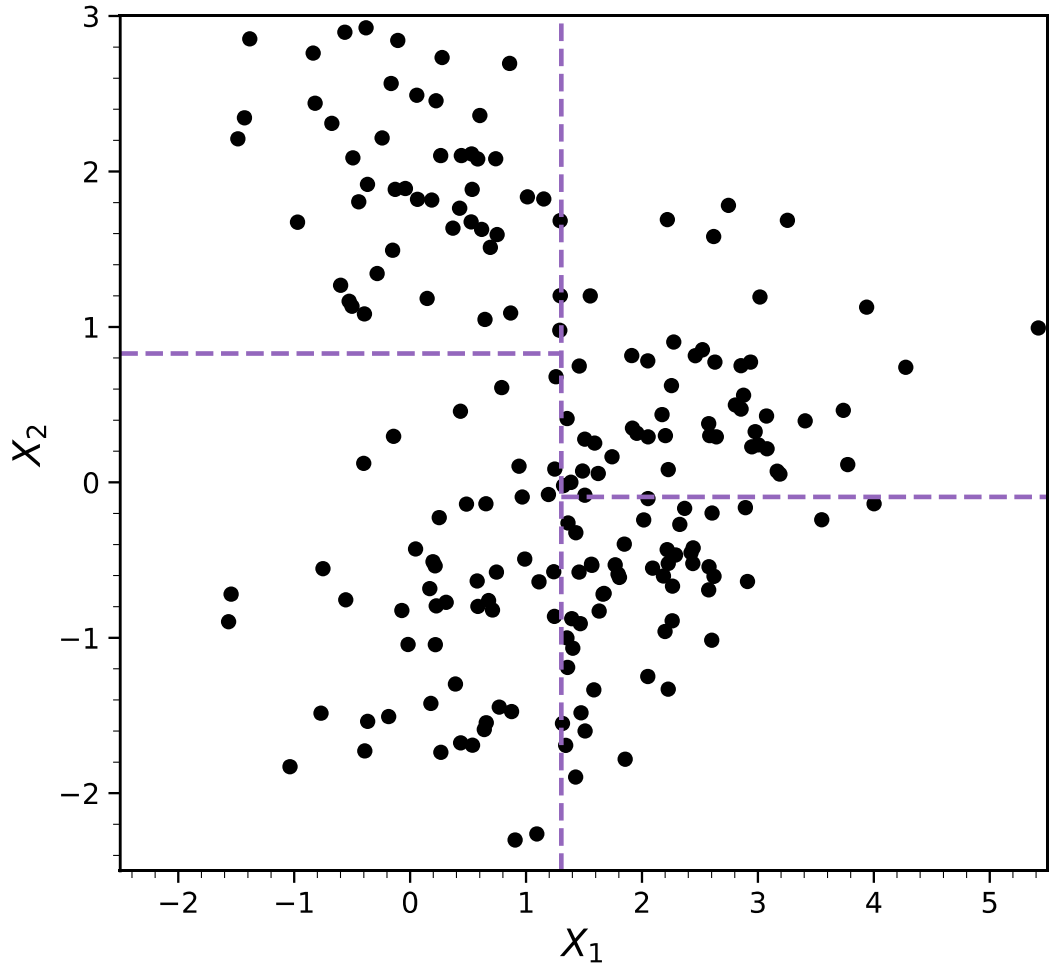
(1) For each point find its K nearest neighbours

(2) Assign the class with most representatives

For N samples, do N(N-1) operations $\mathcal{O}(N^2)$

Can we do better? Yes, with $\mathcal{O}(N \log N)$

Using KD-Trees (aka binary trees)!

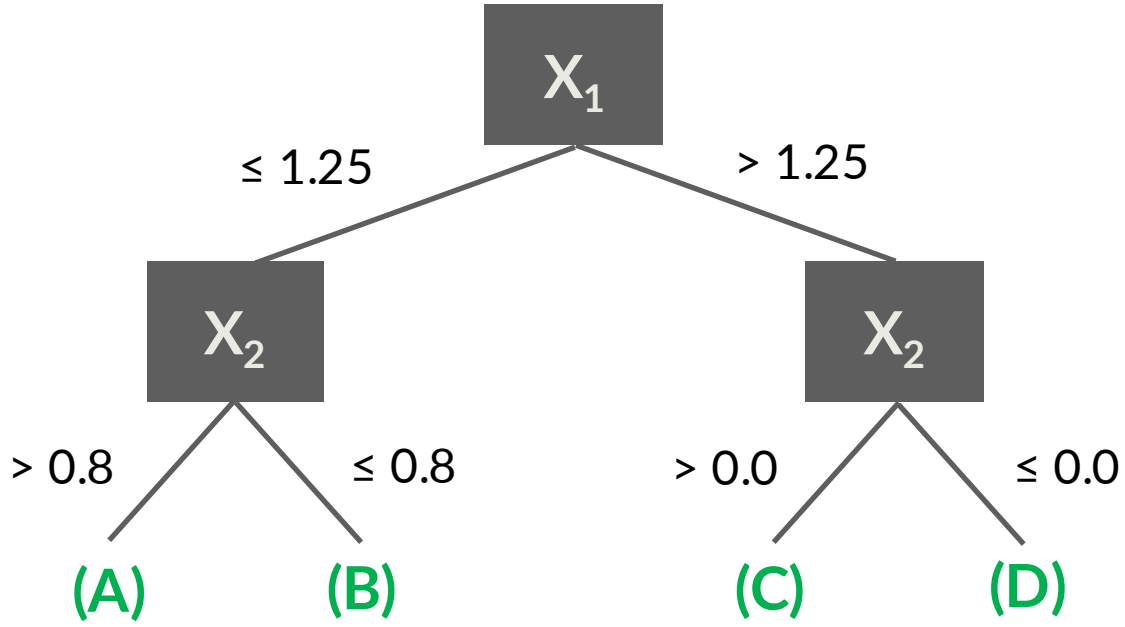


Decision trees

But first, we will talk again about K-nearest neighbours (kNN)

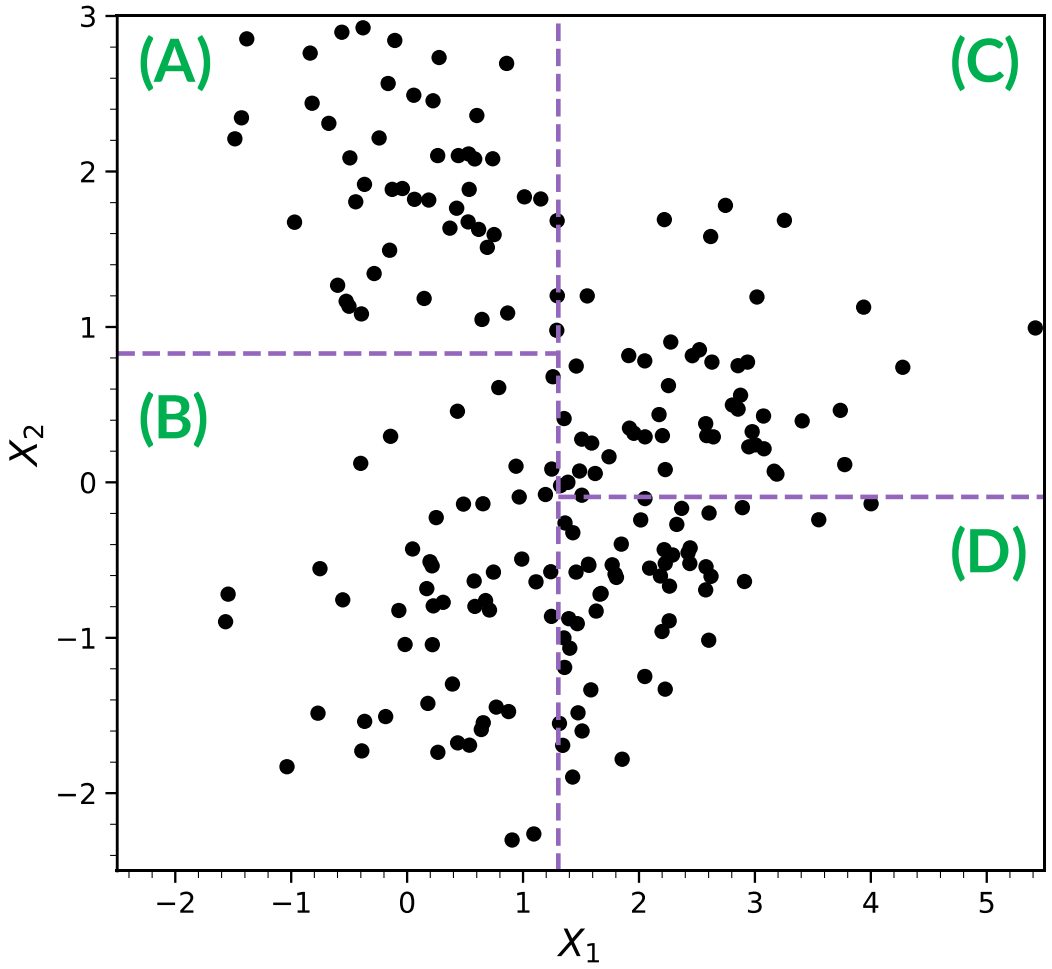
Can we do better? Yes, with $\mathcal{O}(N \log N)$

Using KD-Trees (aka binary trees)!



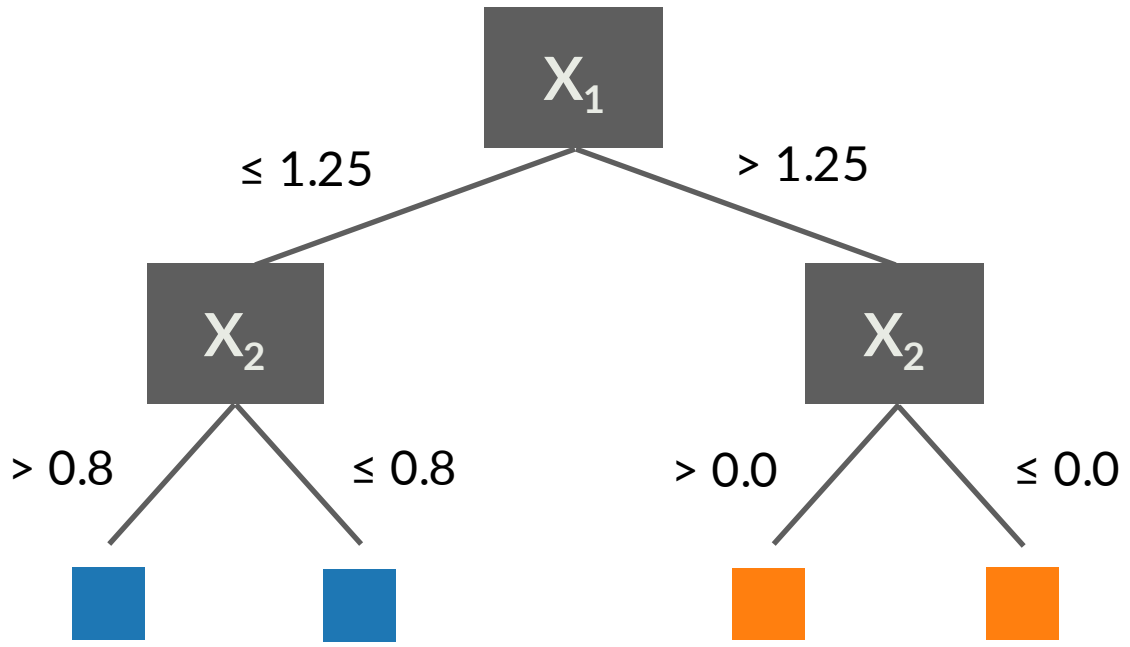
Run kNN in each smaller group of data points

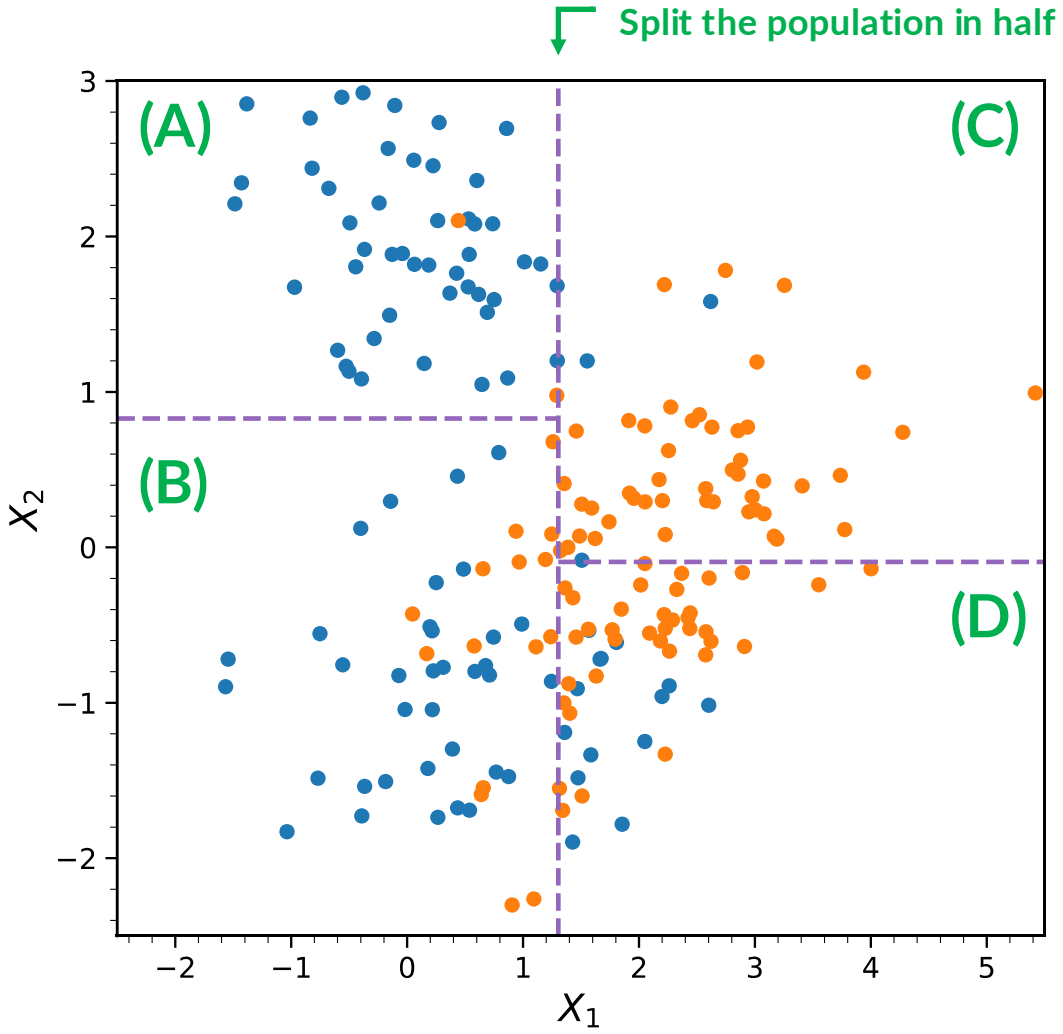
Split the population in half



Decision trees

In **decision trees** we simply assign the majority label to all points in each region



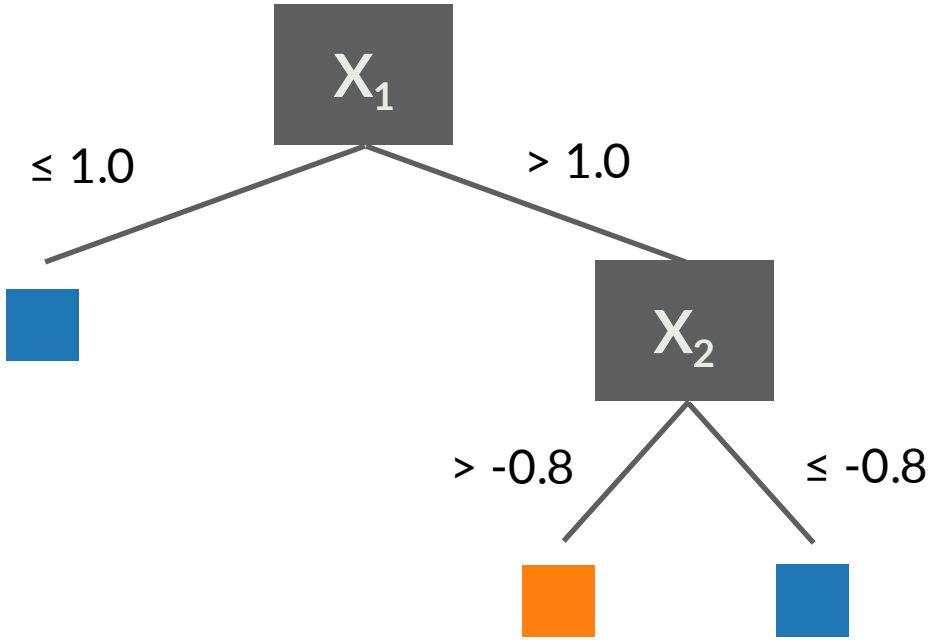


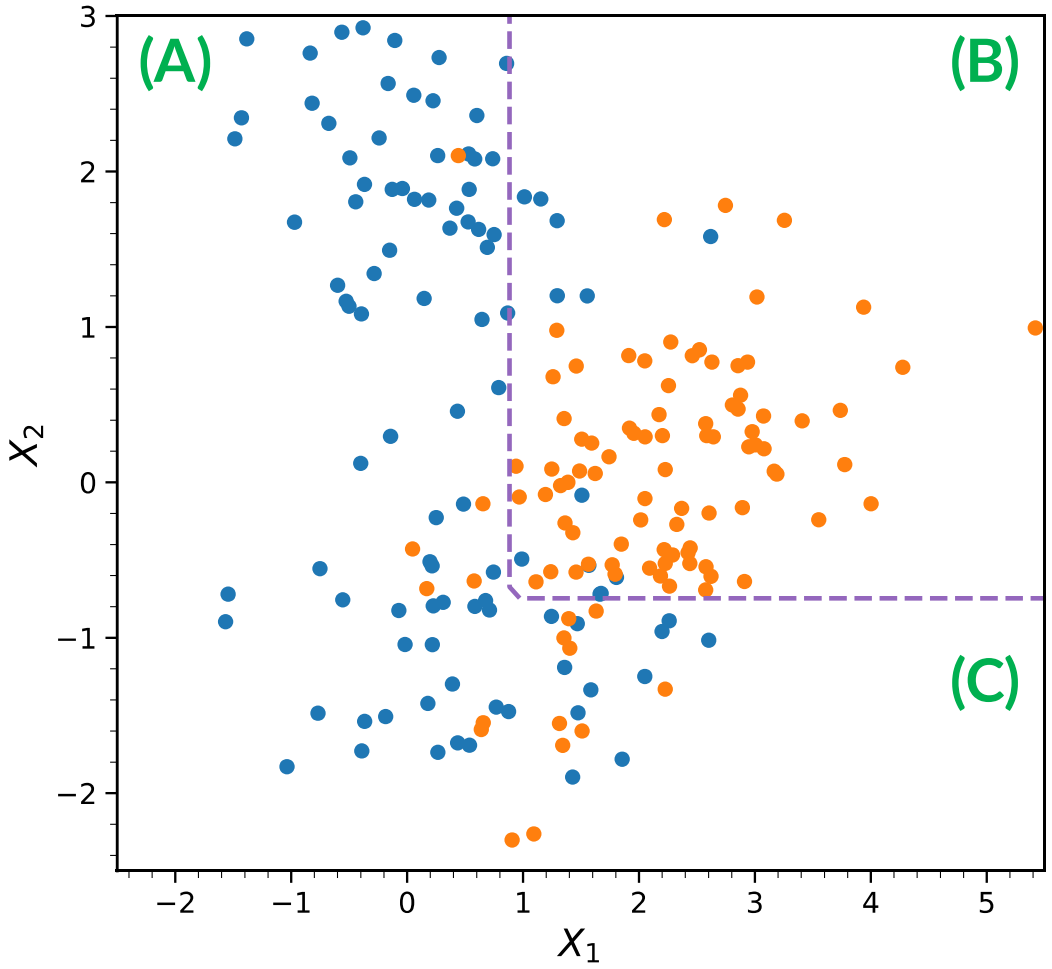
This avoids storing the whole training dataset and repeating nearest-neighbour searches

But maybe the splits should be done differently?

Decision trees

In **decision trees** we simply assign the majority label to all points in each region





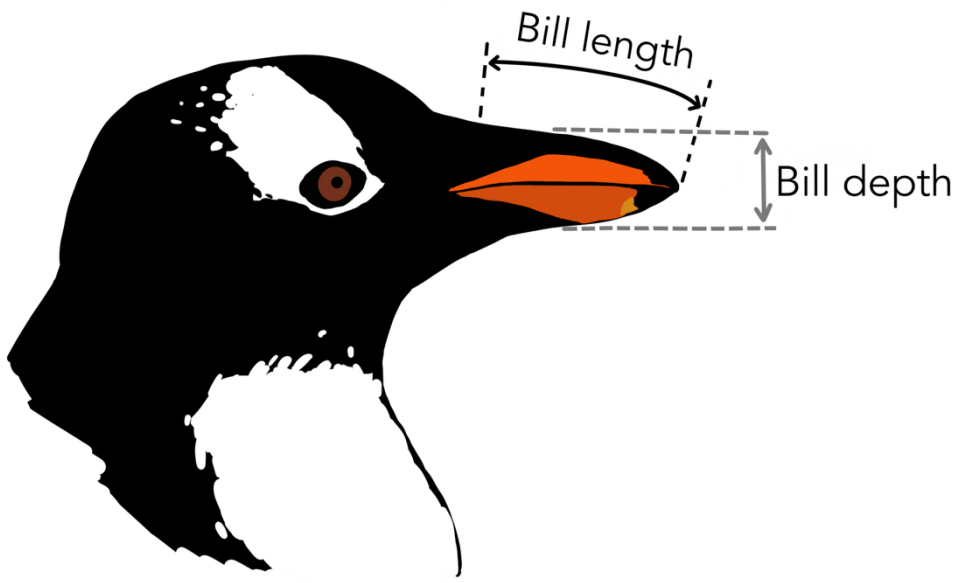
But what is a good criterium to split the data?

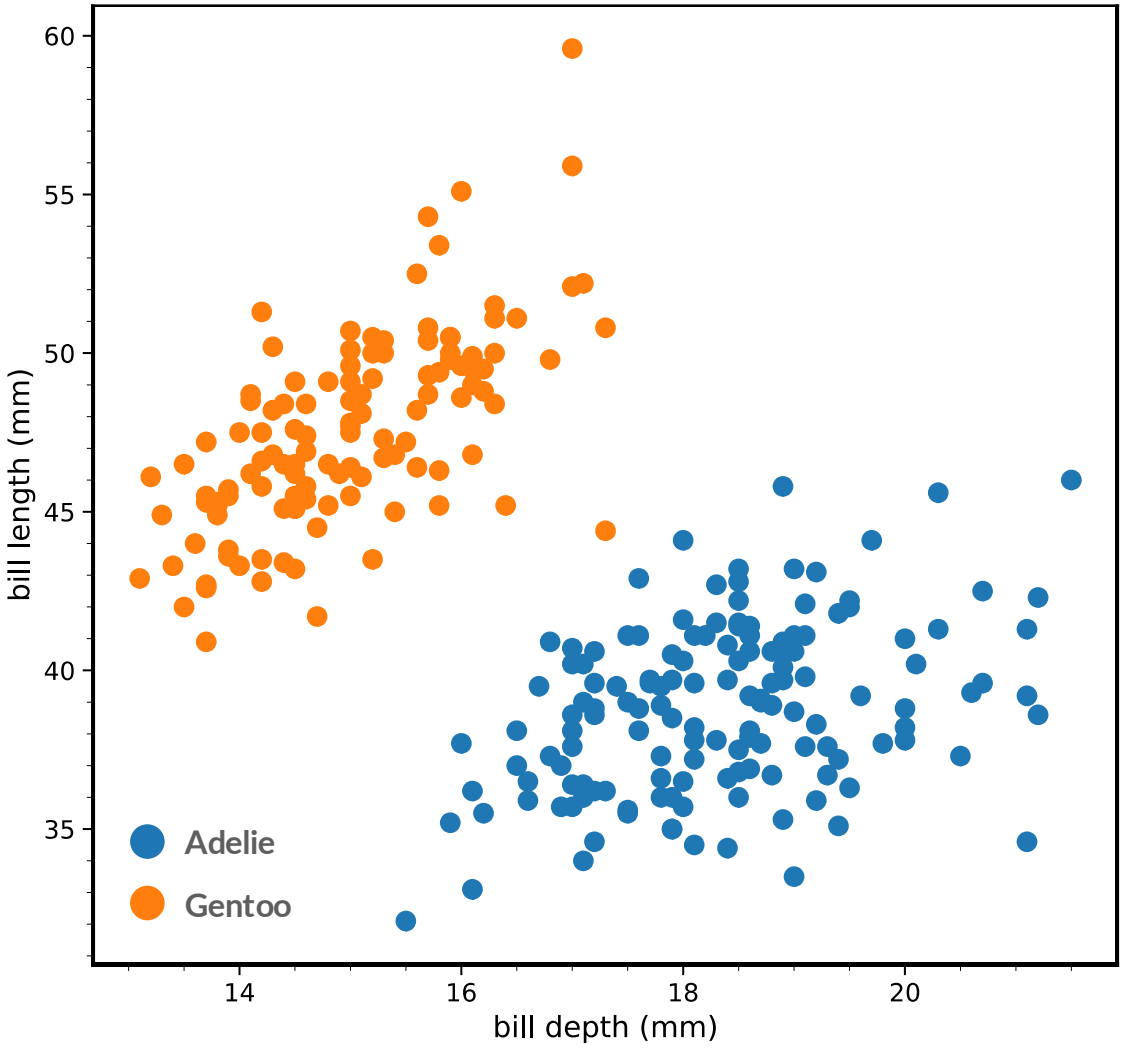
→ Aim for **homogeneous** splits!

Decision trees

When splitting a feature, we want the partitions to be **homogeneous**

Take for example the penguins dataset

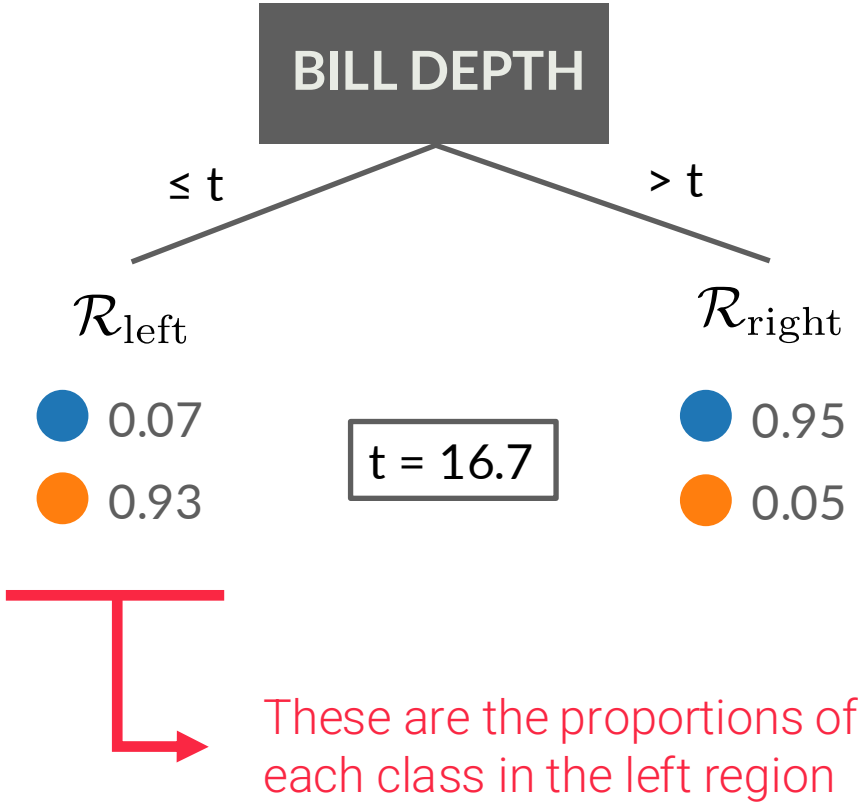


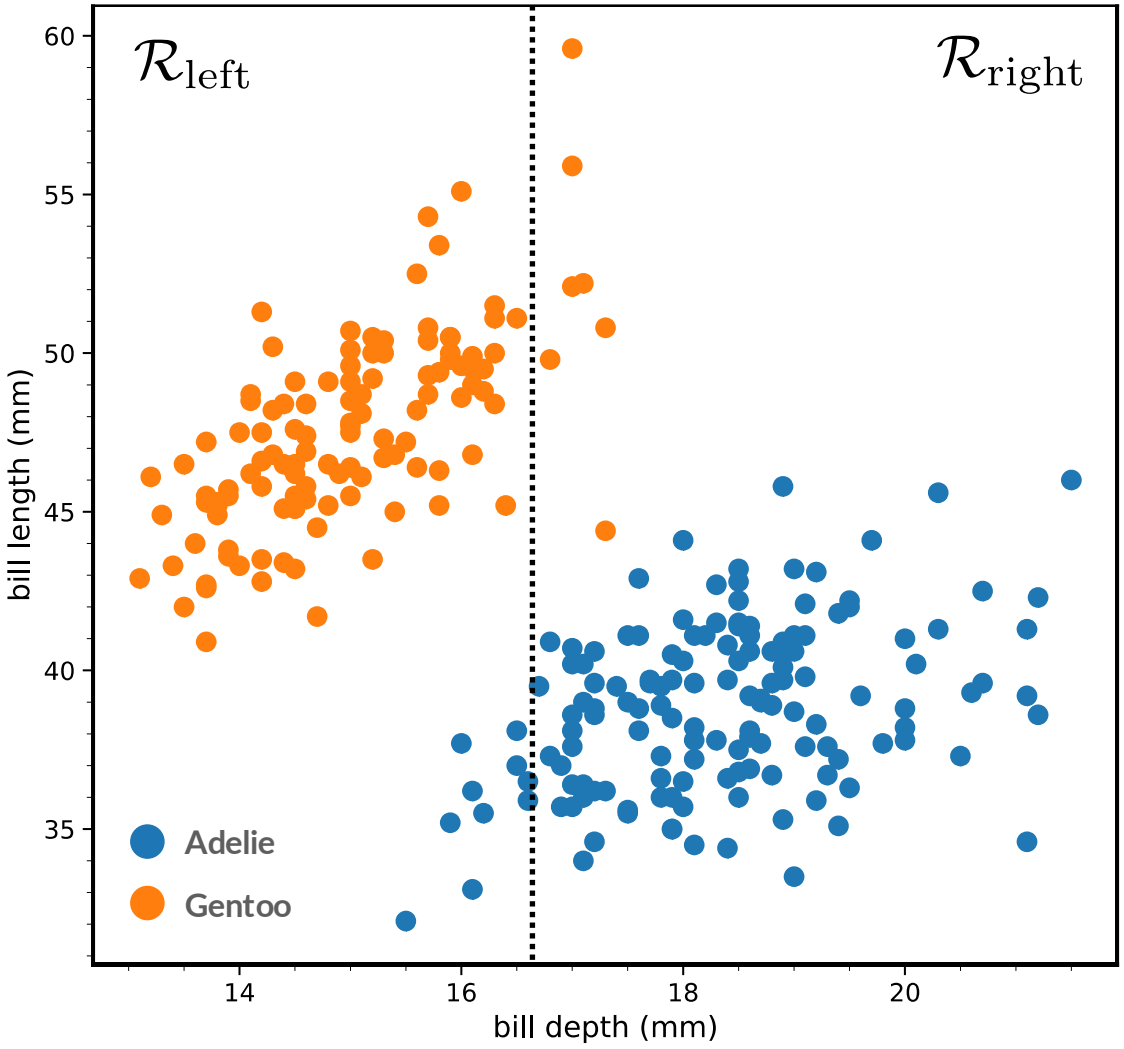


We will look for a split in terms of bill depth

Decision trees

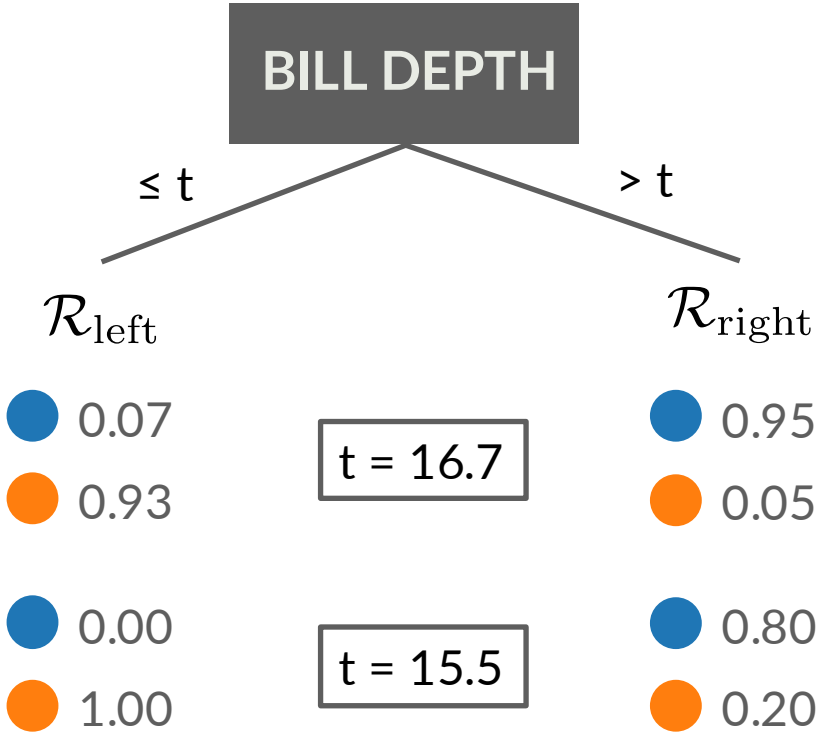
When splitting a feature, we want the partitions to be **homogeneous**

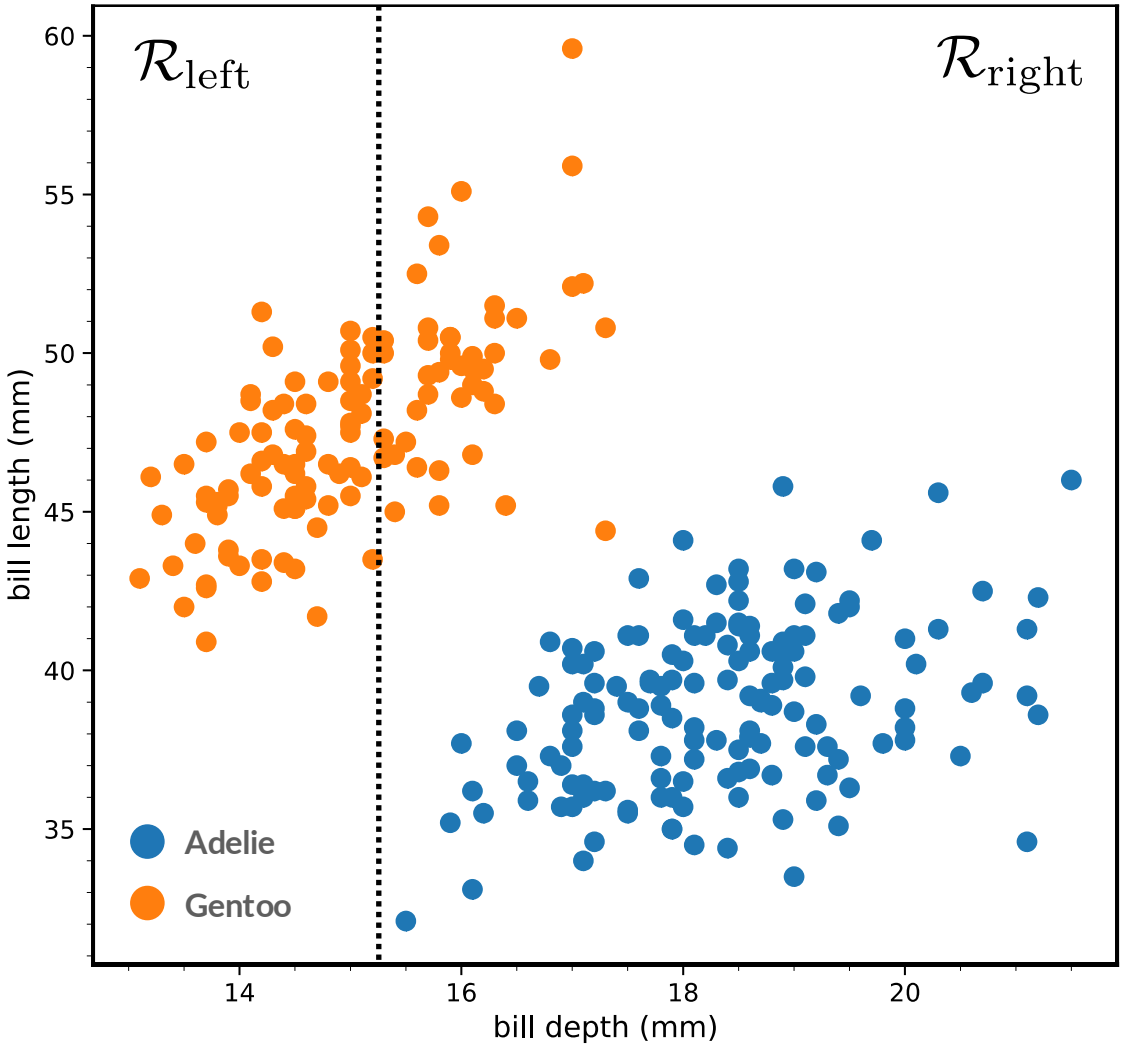




Decision trees

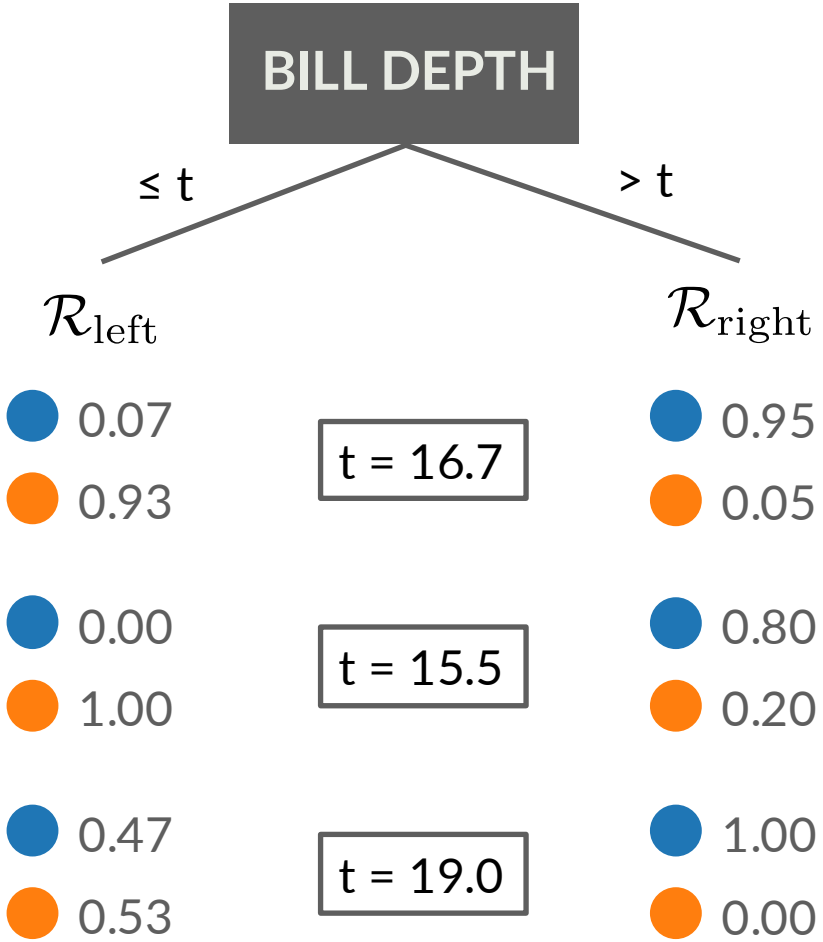
When splitting a feature, we want the partitions to be **homogeneous**

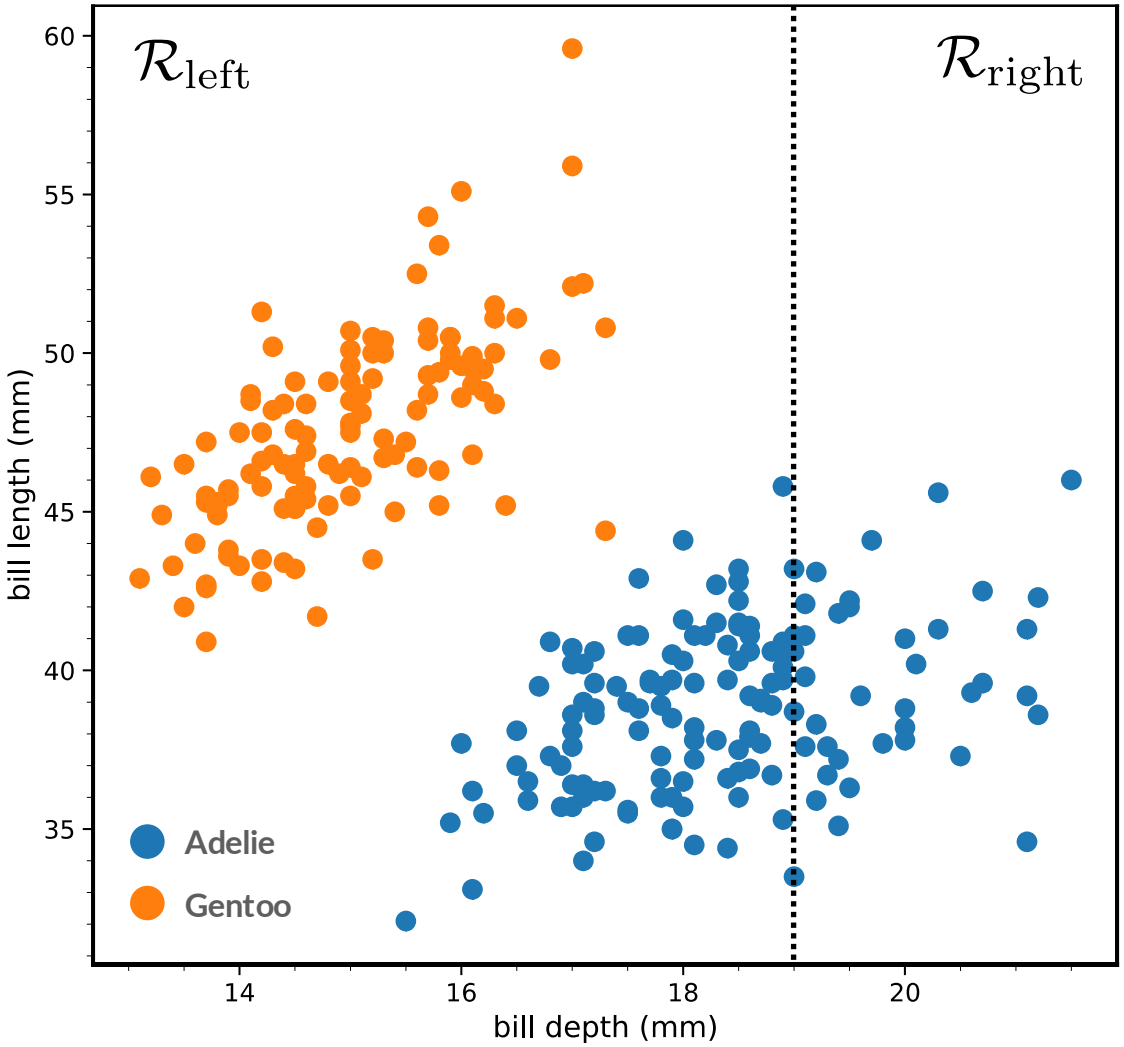




Decision trees

When splitting a feature, we want the partitions to be **homogeneous**





Decision trees

When splitting a feature, we want the partitions to be **homogeneous**

One common way of assessing the homogeneity of a set is via its entropy

For a given dataset $\mathcal{D}$ with classes ■ and ■ we define:

$$H(\mathcal{D}) = -\left(p_{\blacksquare} \log p_{\blacksquare} + p_{\blacksquare} \log p_{\blacksquare}\right)$$

where $p_{\square}$ is the proportion of data points in $\mathcal{D}$ of the class $\square$

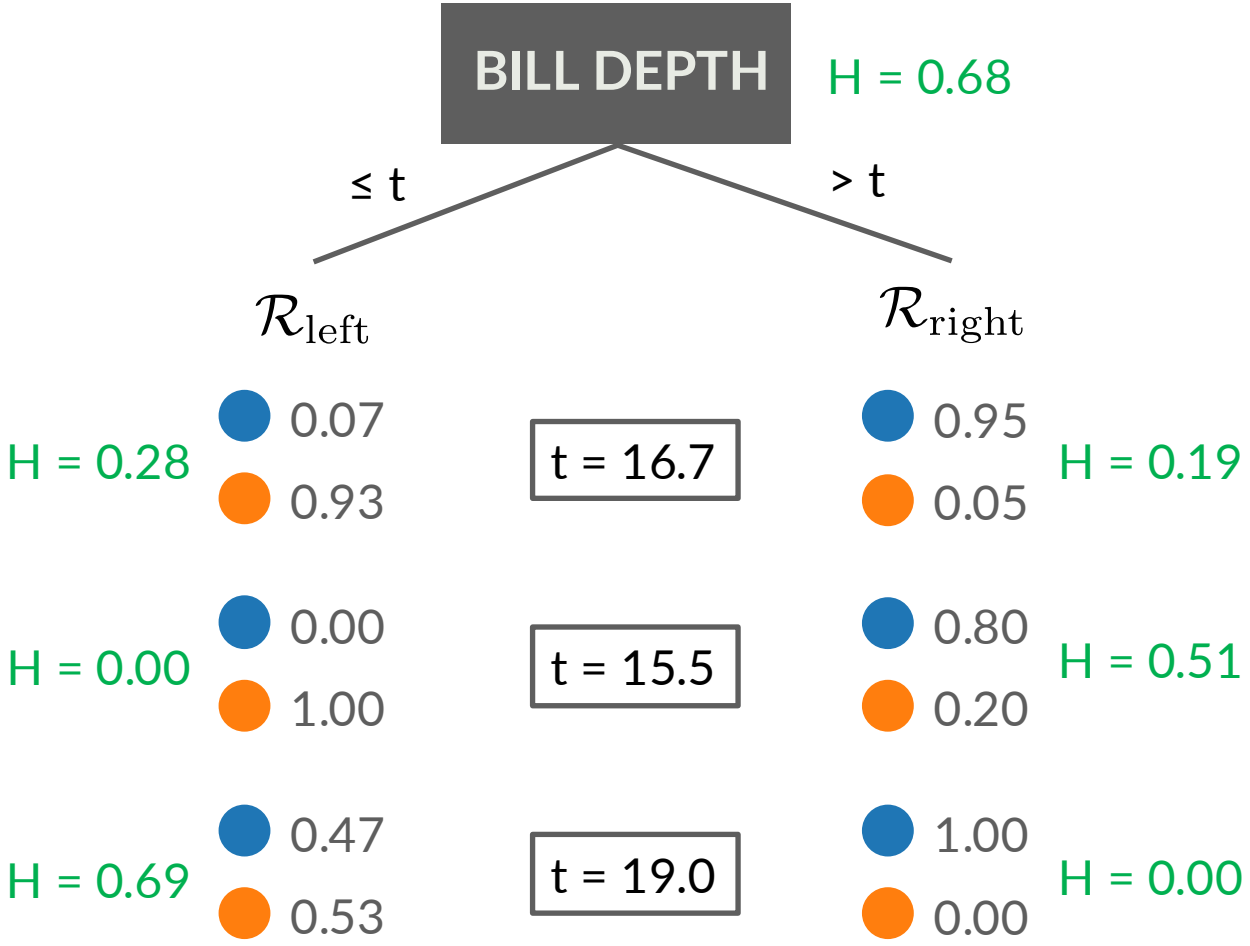
This is a well known measure of disorder used in Physics, Information theory, etc.

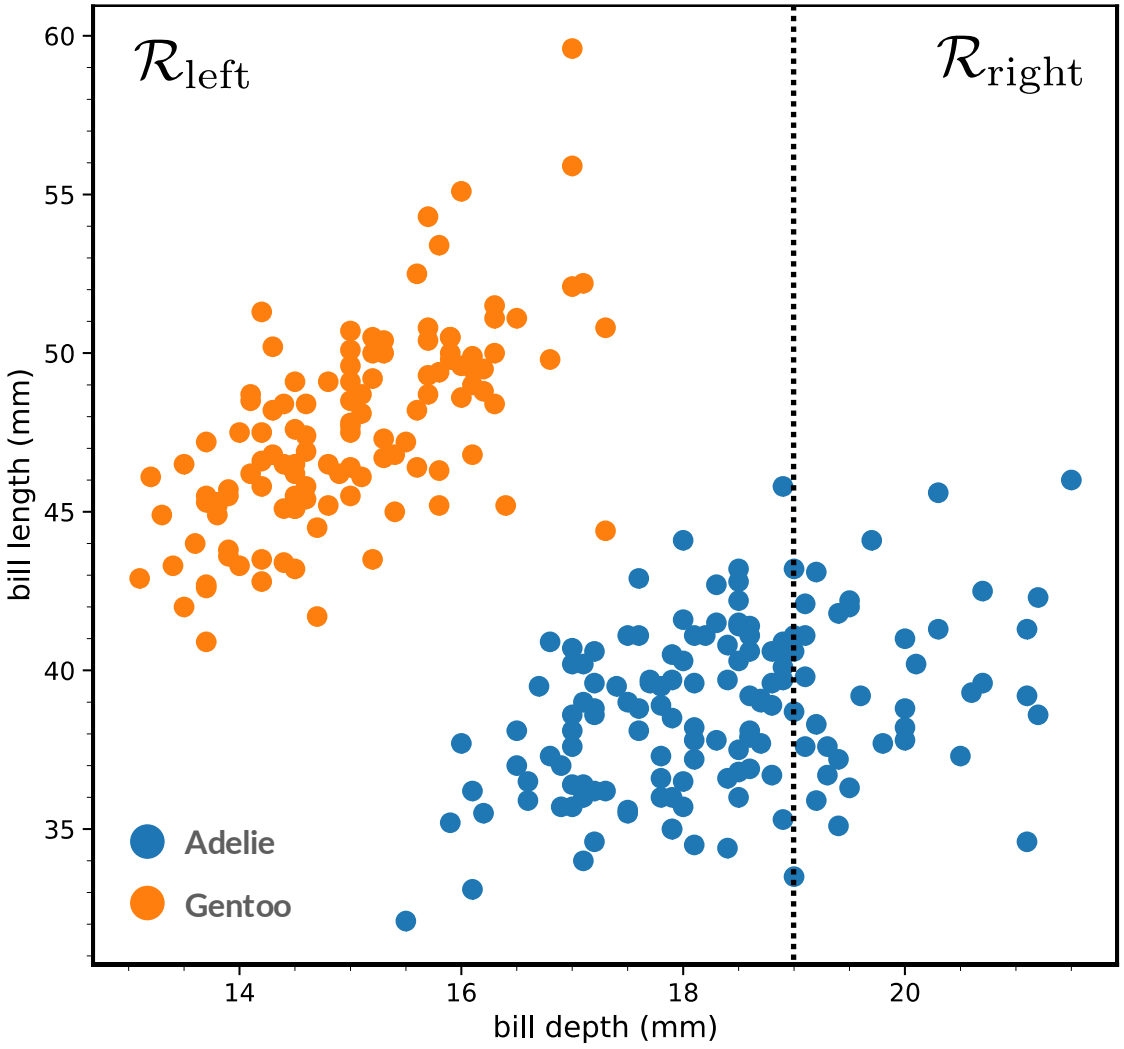
The entropy is maximized when the probabilities for each class are the same. Why?

In the case of decision trees we would like to have splits that have minimum entropy.

Decision trees

When splitting a feature, we want the partitions to be **homogeneous**





Decision trees

When splitting a feature, we want the partitions to be **homogeneous**

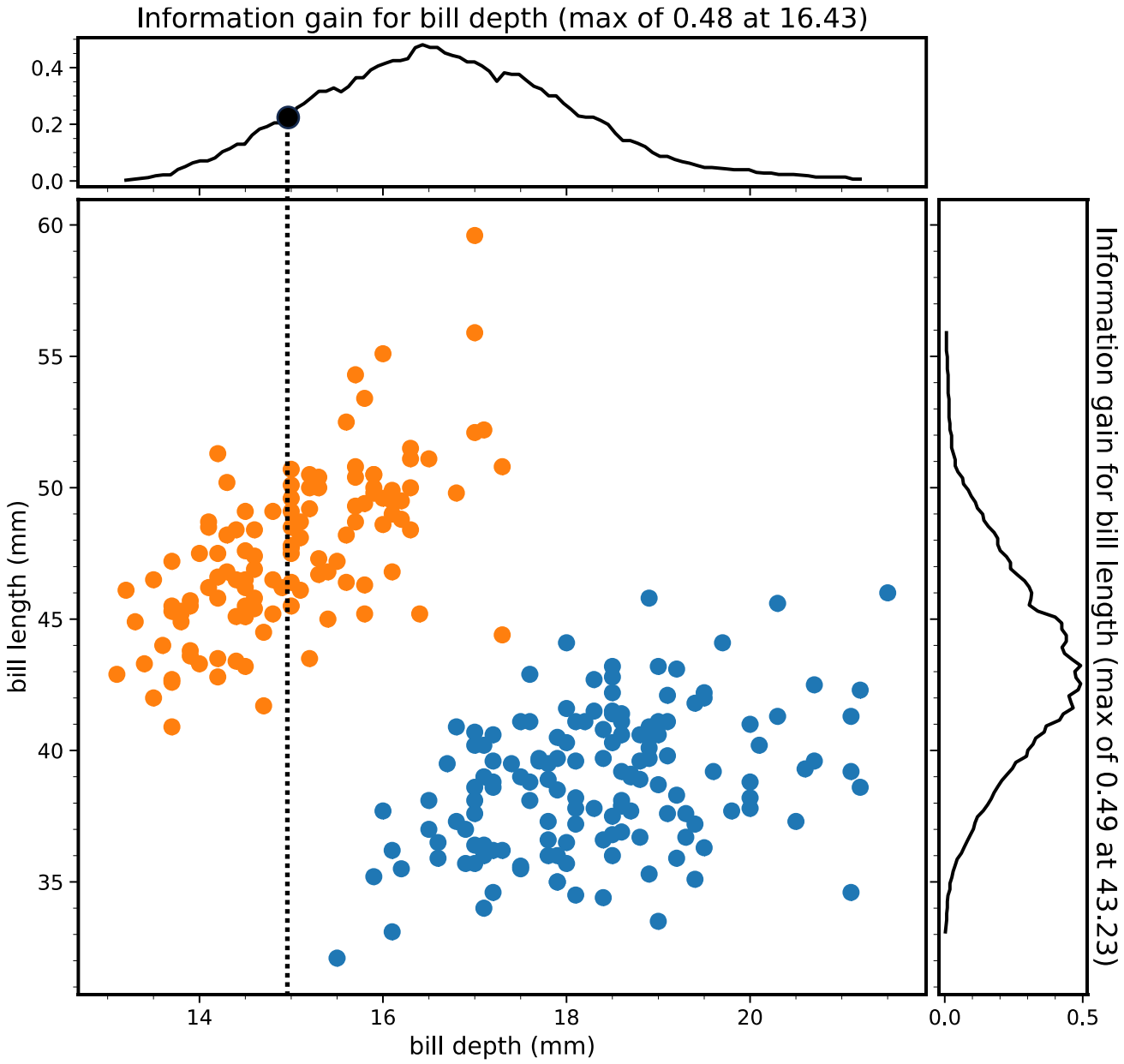
We can build a single criterium based on entropy called information gain

$$I(t) = H(\mathcal{D}) - \underbrace{\frac{|\mathcal{R}_{\text{left}}|}{|\mathcal{D}|}} \times H(\mathcal{R}_{\text{left}}) - \underbrace{\frac{|\mathcal{R}_{\text{right}}|}{|\mathcal{D}|}} \times H(\mathcal{R}_{\text{right}})$$

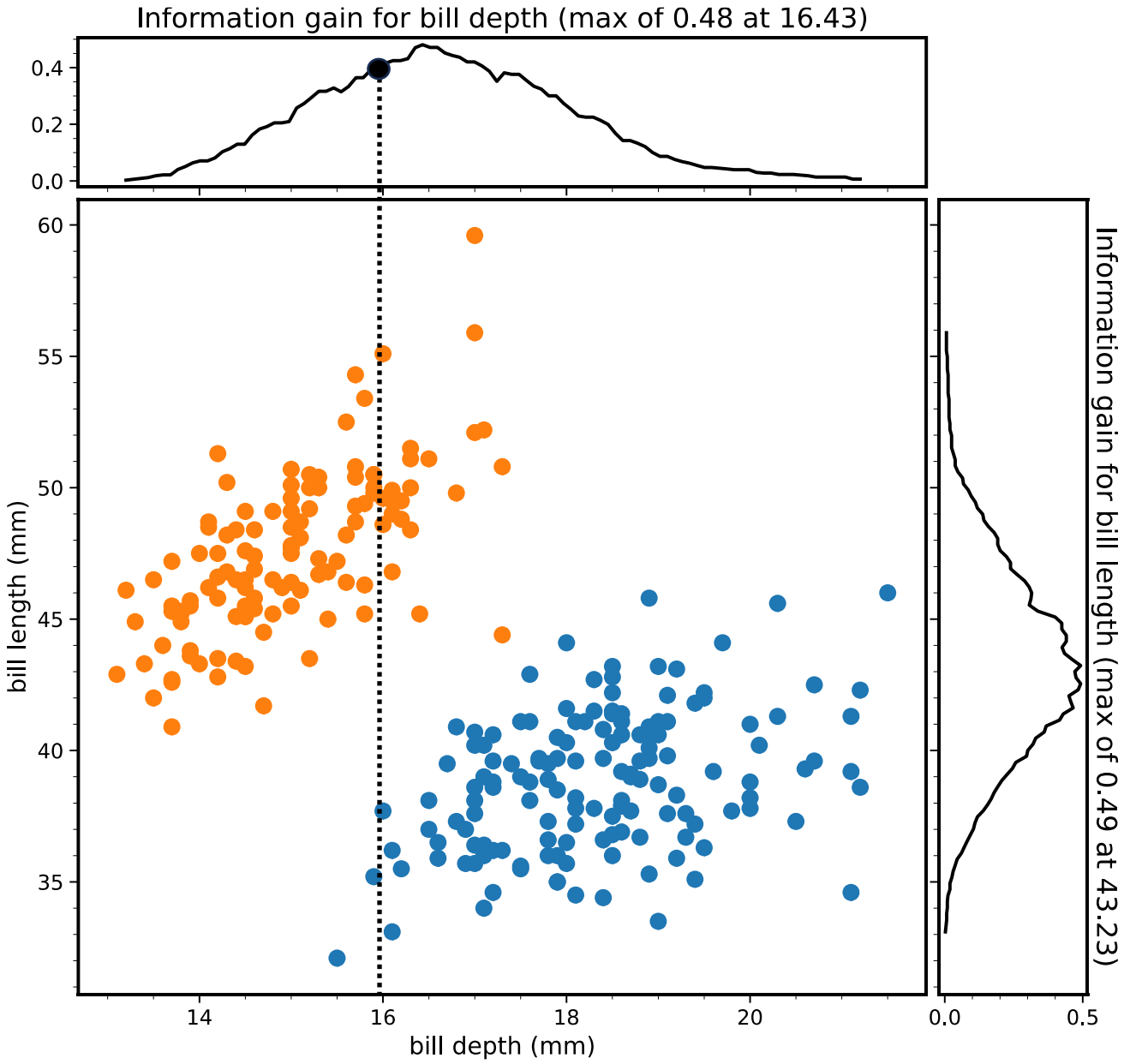
These terms ensure that the entropies are comparable

where the input argument t is the threshold value that defines how to split the dataset

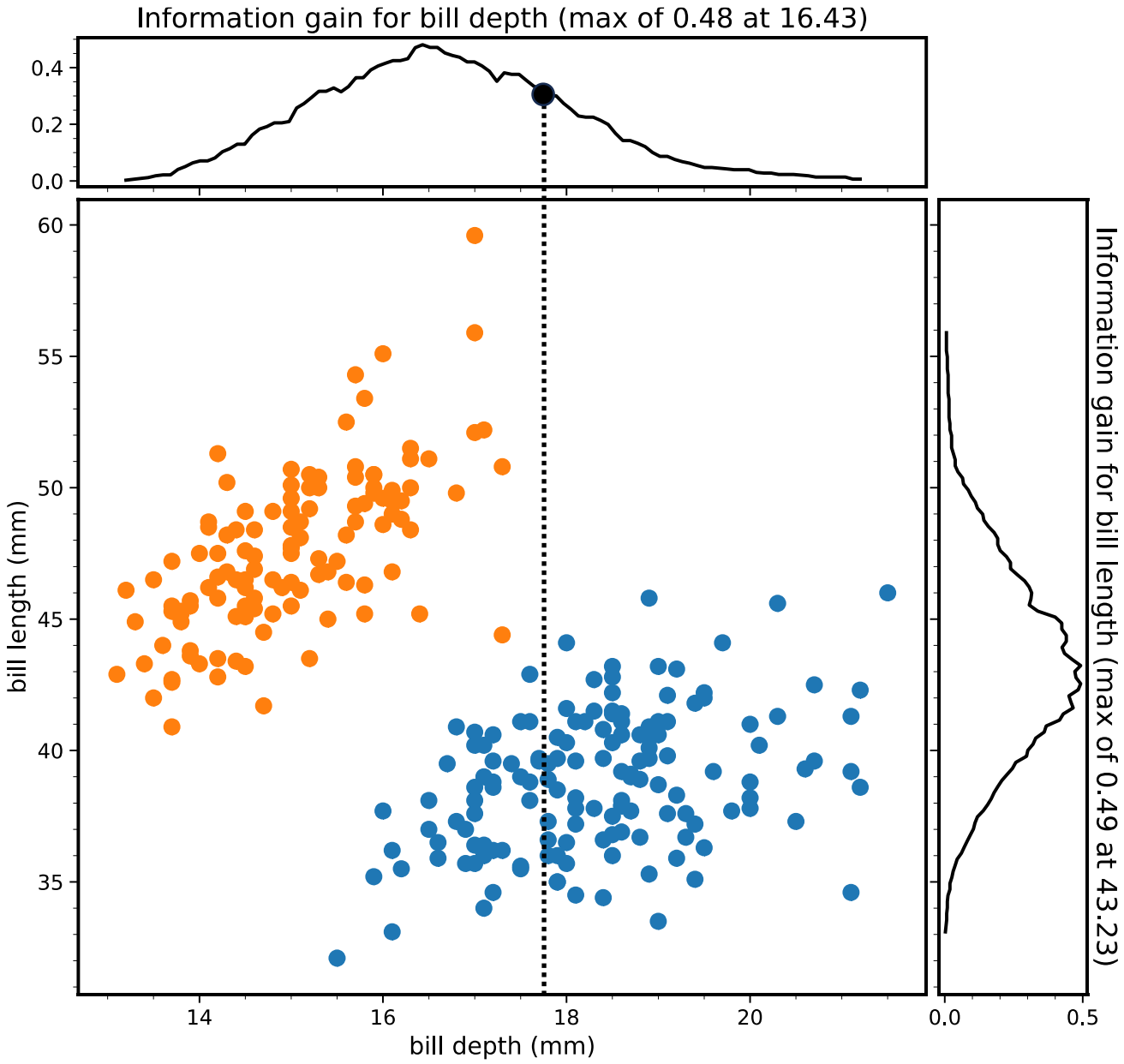
Decision trees



Decision trees



Decision trees



Decision trees

The algorithm for **growing a tree** is therefore:

Start with dataset $\mathcal{D}$ with N points of P dimensions and K classes

For each feature P_i :

For each possible split S_j :

Calculate each one of the K probabilities p_k

Calculate the **entropies** of each side

Get the feature P_i and split S_j that yield **maximum information gain**

Split the dataset according to these values

Continue recursively (stop criterium can be depth of tree, number of points in each leaf, etc)

The algorithm complexity is

$$\mathcal{O}(PNK)$$

but can be improved

Decision trees

Using scikit learn

DecisionTreeClassifier

```
class sklearn.tree.DecisionTreeClassifier(*, criterion='gini',
splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None, random_state=None,
max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None,
ccp_alpha=0.0, monotonic_cst=None)
```

[source]

A decision tree classifier.

Read more in the User Guide.

Note that the default criterion for splitting is based on **gini index**

$$H(\mathcal{R}) = -\sum_{i=1}^{C} p_i \log(p_i) \quad \Longrightarrow \quad G(\mathcal{R}) = 1 - \sum_{i=1}^{C} p_i^2$$

number of classes

Decision trees

Note that a decision tree **classifier** can be written as a sum of indicator functions

$$\boxed{h(x) = \sum_{j=1}^{J} \underline{c(j)} \mathbf{I}(x \in \mathcal{R}_j)}$$

Number of regions in feature space

Class of all data points in region R_j

The same formulation could be used to describe a decision tree **regressor**.

-- Instead of assigning a class to a region, we associate a real number.

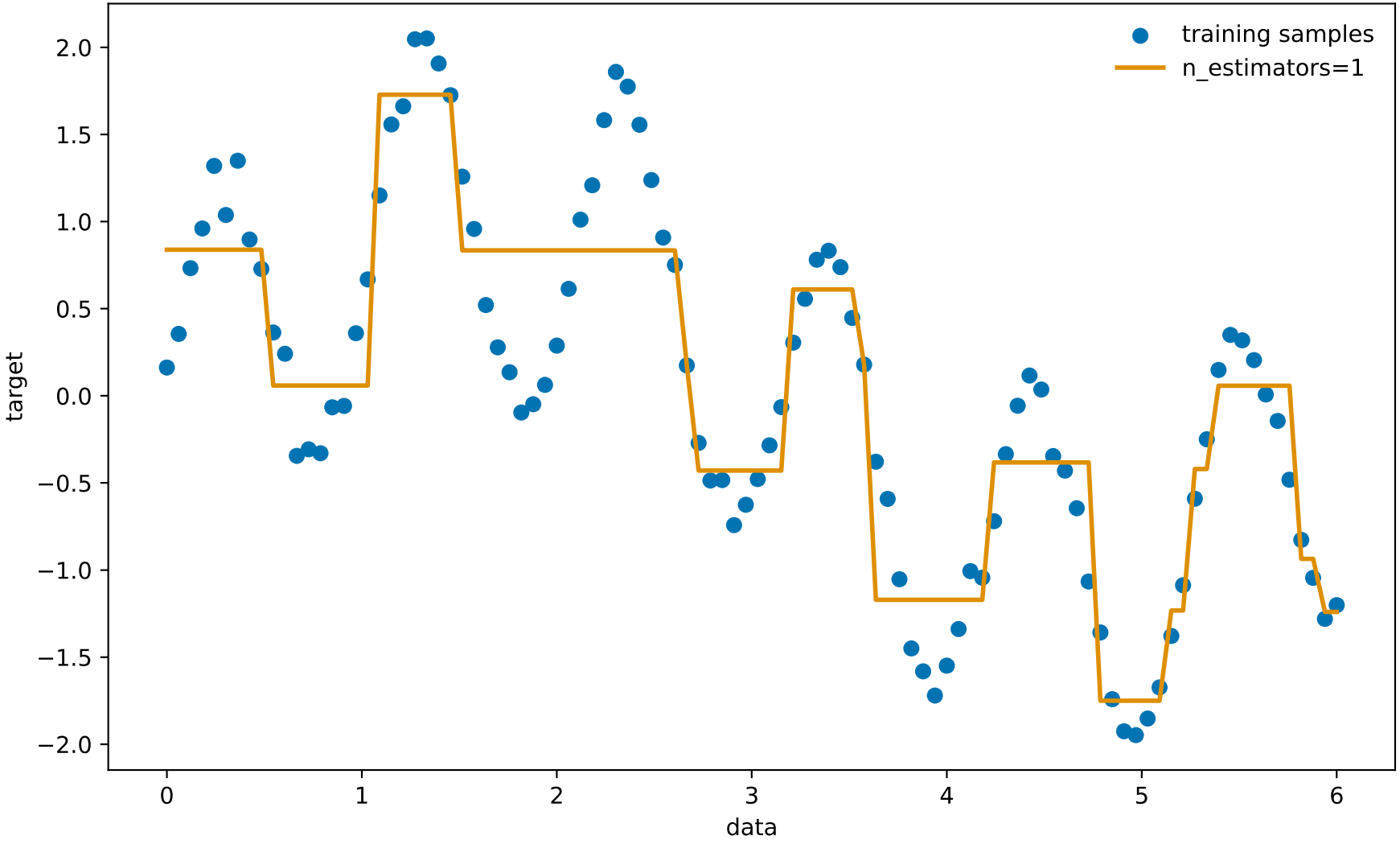
-- However, the splitting criterium must change… though we still aim for **homogeneity**

$$\mathrm{Var}(\mathcal{R}) = \frac{1}{|\mathcal{R}|} \sum_{i:x_i \in \mathcal{R}} (y_i - \hat{y}_{\mathcal{R}})^2$$

Mean of the observations in region

Decision trees

The same formulation could be used to describe a decision tree **regressor**.



Decision trees

Decision trees can also be used for **regression**

$$h(x) = \sum_{j=1}^{J} \hat{y}_{\mathcal{R}_j} \mathbf{I}(x \in \mathcal{R}_j)$$

There are basically two steps involved:

(1) Divide predictor space into J distinct and non-overlapping regions $R_1, \ldots, R_J$

(2) For every observation that falls into the region R_j make the same prediction

For each predictor j and candidate for splitting value s, we define two regions:

$$\mathcal{R}_{\text{left}}(j, s) = \{X | X_j < s\} \qquad \mathcal{R}_{\text{right}}(j, s) = \{X | X_j \geq s\}$$

and calculate $C(j, s) = \frac{|\mathcal{R}_{\text{left}}|}{|\mathcal{D}|} \text{Var}\Big(\mathcal{R}_{\text{left}}(j, s)\Big) + \frac{|\mathcal{R}_{\text{right}}|}{|\mathcal{D}|} \text{Var}\Big(\mathcal{R}_{\text{right}}(j, s)\Big)$

We use the same **recursive binary splitting** from before to construct the regions

Decision trees

The algorithm for **growing a tree** is therefore:

Start with dataset $\mathcal{D}$ with N points of P dimensions and K classes

For each feature P_i :

For each possible split S_j :

Calculate each one of the K probabilities p_k

Calculate the **variances** of each side

Get the feature P_i and split S_j that yield **minimum total variance**

Split the dataset according to these values

Continue recursively (stop criterium can be depth of tree, number of points in each leaf, etc)

The algorithm complexity is

$$\mathcal{O}(PNK)$$

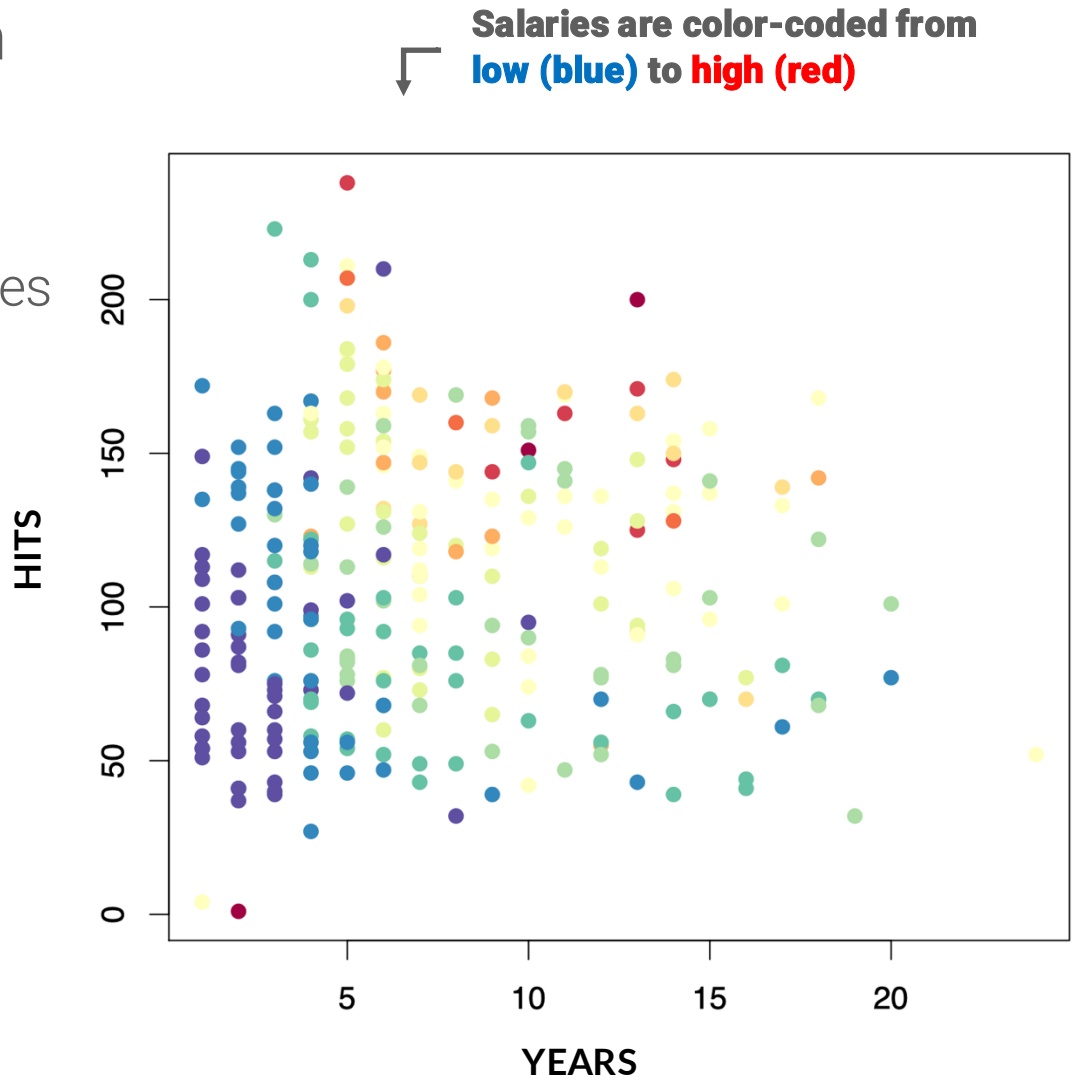
but can be improved

Decision trees

Decision trees can also be used for regression

Take for example the `hitters` dataset in which we try to predict the salary of a baseball player based on

- The number of years that he has played in major leagues
- The number of hits that he made in the previous year



Decision trees

Decision trees can also be used for regression

Take for example the `hitters` dataset in which we try to predict the salary of a baseball player based on

- The number of years that he has played in major leagues and
- The number of hits that he made in the previous year

NB: we took the log of the salaries

Decision trees

In summary, decision trees have many **advantages** vs other approaches we've seen

- Trees are very easy to explain to people, even more than regression!
- Some say that decision trees mirror more closely human decision-making
- Trees can be displayed graphically
- Trees can easily/naturally handle categorical predictors

Decision trees

In summary, decision trees have many **advantages** vs other approaches we've seen

- ▲ Trees are very easy to explain to people, even more than regression!
- ▲ Some say that decision trees mirror more closely human decision-making
- ▲ Trees can be displayed graphically
- ▲ Trees can easily/naturally handle categorical predictors

However, they also have some important **disadvantages**

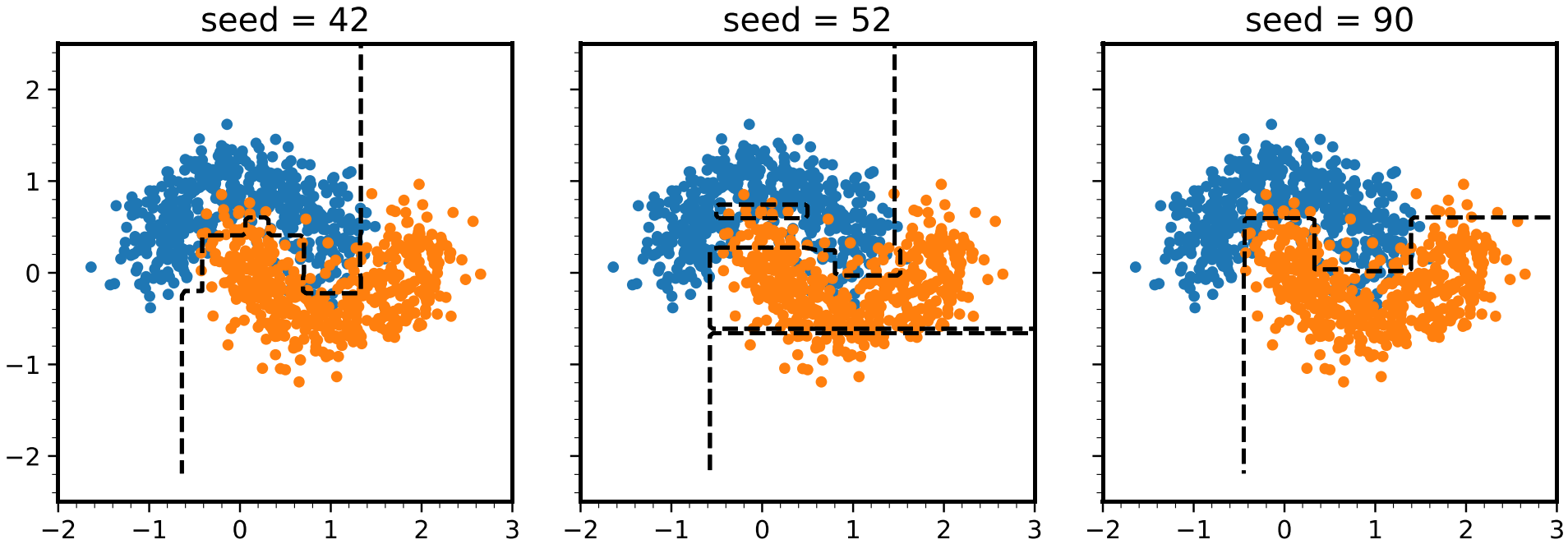
- ▼ They do not have the same level of accuracy as other approaches we've seen
- ▼ Trees can be very non-robust to small changes in the training set

Decision trees

Consider the two moons dataset.

Simulate <u>three different training datasets</u> with 200 points and one test set with 1000.

The scatter dots are points from the test set and the boundary is for a **Decision Tree**



Ensemble methods

Note that the variance of an estimator along different training sets is

$$\mathrm{Var}(f) = \mathbb{E}_{\mathcal{D}}\left[\mathbb{E}_X\left[\left(f_d(X) - \mathbb{E}_X\left[f_d(X)\right]\right)^2 \middle| \mathcal{D} = d\right]\right]$$

Note that we could try to reduce the variance by proposing a **meta-estimator** given by

$$\tilde{f}(X) = \frac{1}{M}\sum_{k=1}^{M} f_{\mathcal{D}_k}(X) \approx \mathbb{E}_D\left[\mathbb{E}_X\left[f_d(X) \middle| \mathcal{D} = d\right]\right]$$

where the $\mathcal{D}_k$ are IID realizations of the distribution generating the dataset

Question: How can we do this in practice if we have a fixed training dataset?

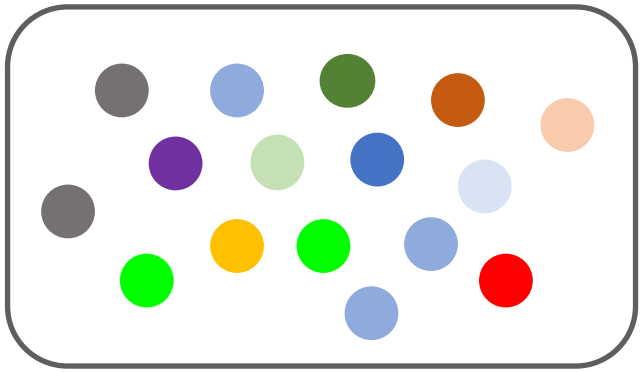
↳ Splitting the dataset would yield IID sub-datasets, but would not work well -- Why ?

Ensemble methods

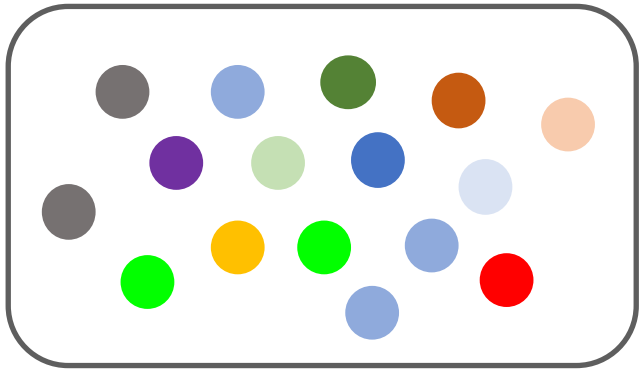
We use bootstrap : generate M datasets by sampling $\mathcal{D}$ with replacement

datasets of the same size

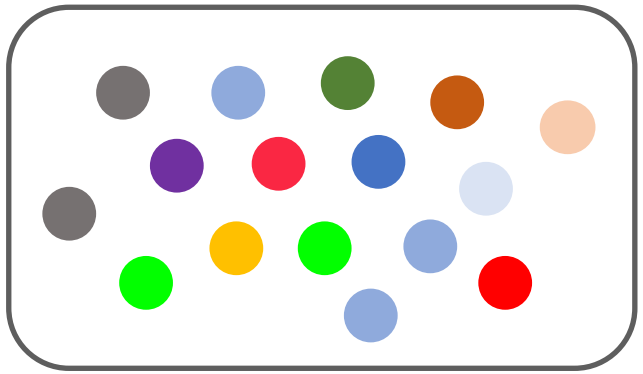
$\mathcal{D} =$



$\mathcal{D}_1 =$



$\cdots \quad \mathcal{D}_M =$



Estimator $\tilde{f}(X) = \frac{1}{M}\sum_{k=1}^{M} f_{\mathcal{D}_k}(X)$ will reduce the variance, but slower than IID case

(Why?)